

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN THỰC HÀNH

Môn: An toàn và bảo mật dữ liệu trong hệ thống thông tin

**ỨNG DỤNG QUẢN TRỊ CSDL ORACLE &
ỨNG DỤNG QUẢN LÝ DỮ LIỆU Y TẾ**

GVHD: TS. Phạm Thị Bạch Huệ

ThS. Lương Vĩ Minh

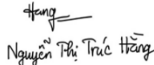
ThS. Tiết Gia Hồng

Lớp: CQ2023/1 - Nhóm: CQ12

Thành phố Hồ Chí Minh, tháng 06, năm 2026

THÔNG TIN NHÓM

- Nhóm: CQ12
- Số lượng thành viên: 5

STT	MSSV	Họ và tên	Vai trò	Chữ ký
1	23120189	Hoàng Quốc Việt	Thành viên	
2	23120193	Trần Kim Yên	Trưởng nhóm	
3	23120201	Nguyễn Thị Trúc Hằng	Thành viên	
4	23120209	Lê Hoàng Nhật Anh	Thành viên	
5	23120237	Lê Lâm Trí Đức	Thành viên	

MỤC LỤC

THÔNG TIN NHÓM.....	1
DANH SÁCH CHỨC NĂNG ĐÃ HOÀN TẤT.....	3
PHÂN CÔNG VÀ ĐÁNH GIÁ	5
1. Phân hệ 1.....	5
2. Phân hệ 2.....	6
NỘI DUNG ĐỒ ÁN	7
1. Phân hệ 1 – Ứng dụng quản trị CSDL Oracle	7
1.1. Giới thiệu phân hệ	7
1.2. Yêu cầu 1 - Quản lý User và Role.....	7
1.3. Yêu cầu 2 - Xem danh sách User và Role.....	9
1.4. Yêu cầu 3 - Cấp phát quyền truy cập	9
1.5. Yêu cầu 4 - Thu hồi quyền	12
1.6. Yêu cầu 5 - Tra cứu quyền	13
1.7. Đánh giá phân hệ.....	14
2. Phân hệ 2 – Ứng dụng quản lý dữ liệu y tế	16
2.1. Giới thiệu phân hệ	16
2.2. Yêu cầu 1 – Giải pháp cấp quyền truy cập.....	17
2.3. Yêu cầu 2 – Oracle Label Security (OLS)	24
2.4. Yêu cầu 3 – Kiểm toán (Audit)	28
2.5. Yêu cầu 4 – Sao lưu và phục hồi.....	37
2.6. Đánh giá phân hệ.....	43
TÀI LIỆU THAM KHẢO	45

DANH SÁCH CHỨC NĂNG ĐÃ HOÀN TẤT

STT	NỘI DUNG	HOÀN TẤT
1	PHẦN 1: ỨNG DỤNG QUẢN TRỊ CSDL ORACLE	X
1.1	Cho phép tạo mới, xóa, sửa (hiệu chỉnh) user hoặc role.	x
1.2	Xem danh sách tài khoản người dùng và role trong hệ thống Oracle DB Server.	x
1.3	Cho phép thực hiện việc cấp quyền:	
	1. Cấp quyền cho user, cấp quyền cho role, cấp role cho user.	x
	2. Quá trình cấp quyền có tùy chọn là có cho phép người được cấp quyền có thể cấp quyền đó cho user/ role khác hay không (có chỉ định WITH GRANT OPTION hay không).	x
	3. Thực hiện cấp quyền trên một số loại đối tượng của CSDL như: table, view, stored procedure, function; quyền select, update phải cho phép phân quyền tính đến mức cột; quyền insert, delete thì không.	x
1.4	Cho phép thu hồi quyền từ user hoặc role.	x
1.5	Xem thông tin về quyền của mỗi user hoặc role trên các đối tượng dữ liệu.	x
2	PHẦN 2: ỨNG DỤNG QUẢN LÝ DỮ LIỆU Y TẾ	X
2.1	Yêu cầu 1: Giải pháp cấp quyền truy cập và cài đặt giao diện	x
	1. Em hãy cài đặt cơ sở dữ liệu và thiết lập tài khoản theo mô tả ở TC#1.	x
	2. Em hãy ép thỏa các chính sách bảo mật liên quan vai trò và “Kỹ thuật viên” và “Bệnh nhân” dùng cơ chế RBAC theo mô tả và cài đặt giao diện cho những người dùng liên quan.	x
	3. Em hãy ép thỏa các chính sách bảo mật liên quan vai trò “Điều phối viên” và “Y sĩ/ Bác sĩ” dùng chế VPD theo mô tả và cài đặt giao diện cho những người dùng liên quan.	x
2.2	Yêu cầu 2: Cơ chế phát tán thông báo dùng OLS và cài đặt giao diện	x
2.3	Yêu cầu 3: Vận dụng cơ chế kiểm toán và cài đặt giao diện	x

	1. Kích hoạt kiểm toán hệ thống.	x
	2. Thực hiện kiểm toán dùng Standard audit: theo dõi hành vi của những user cụ thể trên những đối tượng cụ thể của cơ sở dữ liệu gồm table, view, stored procedure, function, có thiết lập theo dõi các hành vi hiện thành công hay không thành công. Sinh viên tự đề nghị 5 ngữ cảnh khác nhau để thiết lập kiểm toán và kiểm chứng lại nhật ký kiểm toán.	x
	3. Sinh viên có thể dùng Fine-grained Audit hoặc Unified Audit để thực hiện kiểm toán trong các tình huống sau và tạo tình huống để có dữ liệu nhật ký kiểm toán với các hành vi theo yêu cầu.	x
	4. Đọc xuất dữ liệu kiểm toán ở mỗi trường hợp.	x
2.4	Yêu cầu 4: Sao lưu và phục hồi dữ liệu và cài đặt giao diện	x
	1. Tìm hiểu các phương pháp thực hiện sao lưu và phục hồi dữ liệu.	x
	2. Hãy hiện thực các phương pháp đó trên HQT CSDL Oracle.	x
	3. Đánh giá ưu khuyết điểm các phương pháp đã tìm hiểu và thử nghiệm.	x
	4. Kết luận.	x

PHÂN CÔNG VÀ ĐÁNH GIÁ

1. Phân hệ 1

MSSV	Họ Tên	Công việc	% Hoàn thành
23120189	Hoàng Quốc Việt	Cài đặt script SQL yêu cầu 1	100
		Cài đặt giao diện ứng dụng	100
		Đóng góp vào phân hệ 1	20
23120193	Trần Kim Yến	Cài đặt script SQL yêu cầu 3	100
		Kết nối ứng dụng với Oracle, cài đặt các chức năng trên ứng dụng.	100
		Đóng góp vào phân hệ 1	20
23120201	Nguyễn Thị Trúc Hằng	Cài đặt script SQL yêu cầu 2	100
		Cài đặt giao diện ứng dụng	100
		Đóng góp vào phân hệ 1	20
23120209	Lê Hoàng Nhật Anh	Cài đặt script SQL yêu cầu 5, các proc/func hỗ trợ chức năng app.	100
		Quay và chỉnh sửa video demo.	100
		Đóng góp vào phân hệ 1	20
23120237	Lê Lâm Trí Đức	Cài đặt script SQL yêu cầu 4, các proc/func hỗ trợ chức năng app.	100
		Trình bày các báo cáo theo yêu cầu.	100
		Đóng góp vào phân hệ 1	20

2. Phân hệ 2

MSSV	Họ Tên	Công việc	% Hoàn thành
23120189	Hoàng Quốc Việt	Cài đặt script SQL yêu cầu 1 câu 1	100
		Thực hiện yêu cầu 4 câu 1	100
		Cài đặt giao diện ứng dụng	100
		Trình bày lý thuyết PH2 YC3	100
		Quay video demo & test ứng dụng	100
		Đóng góp vào phân hệ 2	20
23120193	Trần Kim Yến	Cài đặt script SQL yêu cầu 3	100
		Kết nối ứng dụng với Oracle, cài đặt các chức năng trên ứng dụng.	100
		Trình bày lý thuyết PH1	100
		Kiểm tra lý thuyết báo cáo & test ứng dụng	100
		Đóng góp vào phân hệ 2	20
23120201	Nguyễn Thị Trúc Hằng	Cài đặt script SQL yêu cầu 1 câu 3	100
		Thực hiện yêu cầu 4 câu 3, 4	100
		Cài đặt giao diện ứng dụng	100
		Trình bày lý thuyết PH2 YC2	100
		Tinh chỉnh format báo cáo & test ứng dụng	100
		Đóng góp vào phân hệ 2	20
23120209	Lê Hoàng Nhật Anh	Cài đặt script SQL yêu cầu 2	100
		Kết nối ứng dụng với Oracle, cài đặt các chức năng trên ứng dụng.	100
		Trình bày lý thuyết PH2 YC1	100
		Edit video demo & test ứng dụng	100
		Đóng góp vào phân hệ 2	20
23120237	Lê Lâm Trí Đức	Cài đặt script SQL yêu cầu 1 câu 2	100
		Cài đặt script SQL yêu cầu 4 câu 2	100
		Cài đặt giao diện ứng dụng	100
		Trình bày lý thuyết PH2 YC4	100
		Trình bày báo cáo hướng dẫn build và chạy chương trình & test ứng dụng	100
		Đóng góp vào phân hệ 2	20

NỘI DUNG ĐỒ ÁN

1. Phân hệ 1 - Ứng dụng quản trị CSDL Oracle

1.1. Giới thiệu phân hệ

Phân hệ 1 hướng đến người quản trị cơ sở dữ liệu (DBA), với mục tiêu chuyển các thao tác quản trị Oracle thường được thực hiện bằng câu lệnh thành một quy trình có giao diện. Hệ thống tập trung vào năm nhóm chức năng: quản lý user và role; liệt kê user và role; cấp quyền; thu hồi quyền; và tra cứu quyền. Cách tiếp cận này giúp người quản trị thao tác nhất quán hơn, đồng thời giảm việc phải ghi nhớ cú pháp DDL/DCL trong quá trình vận hành.

Trong phạm vi đồ án, cơ sở dữ liệu được triển khai trong Pluggable Database PDBHOSX. Oracle mô tả PDB là một tập hợp có thể di chuyển gồm schema, schema object và các cấu trúc liên quan, xuất hiện đối với ứng dụng như một cơ sở dữ liệu riêng. Vì vậy, việc đặt toàn bộ dữ liệu và tài khoản của đồ án trong một PDB riêng giúp giới hạn phạm vi quản trị và tách môi trường HospitalX khỏi các PDB khác trong cùng CDB.

Phân hệ được tổ chức theo mô hình ứng dụng khách – máy chủ. Ứng dụng C# WinForms đảm nhiệm giao diện và thu nhận thao tác từ DBA; lớp truy cập dữ liệu sử dụng Oracle.ManagedDataAccess để mở kết nối và gọi stored procedure; Oracle Database thực thi toàn bộ thao tác quản trị, lưu metadata và trả kết quả về giao diện. ODP.NET là bộ cung cấp dữ liệu chính thức của Oracle cho các ứng dụng .NET, cho phép ứng dụng C# kết nối và khai thác các tính năng của Oracle Database.

1.2. Yêu cầu 1 - Quản lý User và Role

1.2.1. Cơ sở lý thuyết

Trong Oracle, user là danh tính dùng để xác thực và thực hiện thao tác trong cơ sở dữ liệu. Các đối tượng do user sở hữu tạo thành schema có cùng tên với user. Role là một tập hợp quyền được đặt tên và có thể gán cho nhiều user hoặc role khác. Mô hình RBAC gán quyền với vai trò, sau đó gán vai trò cho người dùng; nhờ đó việc quản lý quyền theo nhóm đơn giản hơn so với cấp từng quyền trực tiếp cho từng tài khoản.

Đối với yêu cầu “hiệu chỉnh role”, Oracle không cung cấp thao tác đổi tên role. Trong đồ án, việc hiệu chỉnh role được hiểu theo nghĩa nghiệp vụ: thay đổi tập quyền của role bằng GRANT/REVOKE và thay đổi danh sách thành viên bằng cách cấp hoặc thu hồi role. Đây cũng là cách quản trị role phù hợp với mô hình RBAC.

1.2.2. Giải pháp thực hiện

Các thao tác CREATE USER, ALTER USER, DROP USER, CREATE ROLE và DROP ROLE là DDL nên được đóng gói trong stored procedure và thực thi bằng EXECUTE IMMEDIATE. Dynamic SQL phù hợp khi tên user, role, tablespace hoặc đối tượng chỉ được xác định tại thời điểm chạy. Ứng dụng chỉ truyền tham số và nhận kết quả; phần xây dựng câu lệnh được tập trung tại Oracle.

- Các thủ tục quản lý user và role:

Thủ tục	Chức năng	Cách xử lý chính
sp_CreateUser	Tạo user	Tạo user với mật khẩu, tablespace và quota; cấp CREATE SESSION; ghi họ tên vào THONG TIN_NHAN VIEN.
sp_ChangeUserPassword	Đổi mật khẩu	Thực thi ALTER USER ... IDENTIFIED BY ...
sp_LockUser / sp_UnlockUser	Khóa hoặc mở khóa	Thực thi ALTER USER ... ACCOUNT LOCK/UNLOCK.
sp_UpdateUserInfo	Cập nhật hồ sơ hiển thị	Cập nhật họ tên, giới tính, ngày sinh, số điện thoại, địa chỉ và vai trò trong bảng THONG TIN_NHAN VIEN.
sp_DropUser	Xóa user	DROP USER ... CASCADE để xóa tài khoản cùng các object thuộc schema.
sp_CreateRole / sp_DropRole	Tạo hoặc xóa role	Thực thi CREATE ROLE hoặc DROP ROLE bằng dynamic SQL.

- + Procedure sp_CreateUser đồng thời xử lý hai lớp dữ liệu. Lớp thứ nhất là tài khoản Oracle, bao gồm username, password, tablespace và quyền đăng nhập. Lớp thứ hai là hồ sơ phục vụ giao diện, hiện được lưu trong THONG TIN_NHAN VIEN. Cách tách này giúp giao diện có thể hiển thị họ tên và thông tin cá nhân mà không phải mở rộng các view hệ thống của Oracle.
- + Khi xóa user, tùy chọn CASCADE bảo đảm các schema object do user sở hữu cũng bị xóa. Vì đây là thao tác không thể hoàn tác bằng ROLLBACK, ứng dụng cần hiển thị xác nhận rõ ràng trước khi thực thi. Đối với dữ liệu demo, nhóm tạo hai tài khoản BS_AN và YT_BINH, đồng thời bổ sung thông tin cá nhân và dữ liệu lương để kiểm thử phân quyền trên table, view, procedure và function.

1.2.3. Kết quả đạt được

Ứng dụng đã bao phủ vòng đời chính của tài khoản: tạo, đổi mật khẩu, khóa, mở khóa và xóa. Đối với role, hệ thống hỗ trợ tạo, xóa, xem thành viên và hiệu chỉnh nội dung role thông qua các chức năng cấp/thu hồi quyền. Việc đưa các thao tác vào giao diện giúp DBA thực hiện đúng trình tự, đồng thời bảo đảm mọi thao tác quản trị đều đi qua cùng một lớp xử lý tại CSDL.

1.3. Yêu cầu 2 - Xem danh sách User và Role

1.3.1. Cơ sở lý thuyết

Oracle Data Dictionary chứa metadata về user, role, đối tượng và quyền. Các view có tiền tố DBA_ cho phép tài khoản quản trị xem thông tin trên toàn cơ sở dữ liệu. Trong phân hệ này, DBA_USERS được dùng để lấy danh sách user và trạng thái tài khoản; DBA_ROLES cung cấp danh sách role; DBA_ROLE_PRIVS cho biết quan hệ giữa user/role và role được cấp.

1.3.2. Giải pháp thực hiện

USP_LIST_USERS trả kết quả bằng SYS_REFCURSOR để ứng dụng có thể nạp trực tiếp vào bảng dữ liệu. Truy vấn sử dụng ORACLE_MAINTAINED = 'N' nhằm loại các tài khoản do Oracle duy trì, đồng thời loại ADMINHOS, PDBADMIN và common user có tiền tố C##. USP_LIST_ROLES áp dụng nguyên tắc tương tự đối với role. Cách lọc này giúp giao diện tập trung vào các tài khoản và role do nhóm hoặc DBA tạo.

Ngoài danh sách cơ bản, script còn cung cấp USP_LIST_ROLE_MEMBERS để xem thành viên của một role, USP_GET_USER_ROLE để xem role của một user và USP_GET_USER_INFO để lấy hồ sơ hiển thị. Các thủ tục này tách phần truy vấn metadata khỏi lớp giao diện, giúp mã C# chỉ tập trung vào thao tác và trình bày dữ liệu.

1.3.3. Kết quả đạt được

DBA có thể xem danh sách user và role do người quản trị tạo, tra cứu trạng thái tài khoản, tablespace mặc định, thời điểm tạo và thành viên của role. Kết quả trả về theo SYS_REFCURSOR phù hợp với DataGridView và có thể tái sử dụng cho tìm kiếm, phân trang hoặc lọc trên giao diện.

1.4. Yêu cầu 3 - Cấp phát quyền truy cập

1.4.1. Phân loại quyền và ràng buộc

Oracle phân biệt system privilege, object privilege và role. System privilege cho phép thực hiện một loại thao tác ở phạm vi hệ thống hoặc schema, chẳng hạn CREATE SESSION

hoặc CREATE TABLE. Object privilege gắn với một đối tượng cụ thể, chẳng hạn SELECT trên table, UPDATE trên view hoặc EXECUTE trên procedure/function. Role là tập hợp quyền có thể gán cho user hoặc role khác.

SP_GRANT_PRIVILEGE nhận loại quyền, loại đối tượng, tên đối tượng, danh sách cột, grantee, loại grantee và tùy chọn cấp lại. Trước khi tạo câu lệnh, procedure kiểm tra tính hợp lệ của tổ hợp quyền – đối tượng. Việc kiểm tra sớm giúp ngăn các trường hợp như cấp EXECUTE cho table hoặc cấp SELECT cho procedure.

- Danh sách quyền được hỗ trợ trong ứng dụng:

Loại đối tượng	Quyền hỗ trợ	Mức cột	Cơ chế hiện thực
TABLE	SELECT, INSERT, UPDATE, DELETE	SELECT, UPDATE	SELECT mức cột dùng VPD; UPDATE mức cột dùng cú pháp Oracle; INSERT/DELETE ở mức toàn bảng.
VIEW	SELECT, INSERT, UPDATE, DELETE	SELECT, UPDATE	SELECT mức cột dùng view trung gian; UPDATE mức cột dùng cú pháp Oracle khi view cho phép cập nhật.
PROCEDURE	EXECUTE	Không áp dụng	GRANT EXECUTE ON procedure.
FUNCTION	EXECUTE	Không áp dụng	GRANT EXECUTE ON function.
ROLE	Gán role	Không áp dụng	GRANT role TO user/role [WITH ADMIN OPTION].

Theo cú pháp GRANT của Oracle 21c, danh sách cột chỉ được chỉ định trực tiếp với INSERT, REFERENCES hoặc UPDATE; SELECT không hỗ trợ cấp trực tiếp theo cột. Vì đề bài yêu cầu SELECT mức cột, nhóm phải xây dựng cơ chế bổ sung thay vì sử dụng một câu GRANT SELECT(column).

1.4.2. Tùy chọn **WITH GRANT OPTION** và **WITH ADMIN OPTION**

WITH GRANT OPTION áp dụng cho object privilege được cấp cho user hoặc PUBLIC, cho phép người nhận cấp lại quyền đó cho user hoặc role khác. Oracle không cho phép WITH GRANT OPTION khi grantee là role. Vì vậy, script chỉ nối tùy chọn này khi p_grantee_type = USER; nếu grantee là role, procedure bỏ qua và ghi cảnh báo.

Khi cấp role, SP_GRANT_ROLE_TO_USER sử dụng WITH ADMIN OPTION. Tùy chọn này cho phép người nhận tiếp tục cấp hoặc thu hồi role. Hai tùy chọn có ý nghĩa tương tự về khả năng ủy quyền nhưng áp dụng cho hai loại đối tượng khác nhau; giao diện cần thể hiện đúng ngữ cảnh để tránh nhầm lẫn.

1.4.3. SELECT mức cột trên TABLE bằng VPD

Đối với table, nhóm sử dụng Oracle Virtual Private Database (VPD) với cơ chế column masking. DBMS_RLS.ADD_POLICY cho phép chỉ định các cột nhạy cảm bằng SEC_RELEVANT_COLS.

Giải pháp gồm bảng VPD_COL_TRACKING, hàm policy_select_column và procedure SP_GRANT_PRIVILEGE. VPD_COL_TRACKING lưu schema, table, policy, grantee và danh sách cột bị ẩn. Khi người dùng truy vấn, policy_select_column lấy SESSION_USER; nếu user có bản ghi tracking tương ứng, hàm trả predicate 1=2. Kết hợp với ALL_ROWS, predicate này không loại dòng mà làm các cột trong SEC_RELEVANT_COLS bị che thành NULL.

- Tổng quan các bước thực hiện:
 - + Cấp SELECT trên toàn table cho user để Oracle cho phép truy vấn đối tượng.
 - + Chuẩn hóa danh sách cột được cấp và hợp nhất với các cột đã được cấp trước đó.
 - + Tính hidden_cols bằng tập tất cả cột trừ tập cột được phép.
 - + Xóa policy cũ nếu tồn tại và tạo lại policy với hidden_cols mới.
 - + MERGE bản ghi vào VPD_COL_TRACKING để hỗ trợ cấp bổ sung hoặc thu hồi về sau.

Ví dụ, nếu user BS_AN chỉ được phép đọc HOTEN trên THONG TIN_NHAN VIEN, hệ thống cấp SELECT trên bảng và tạo policy che các cột còn lại. Khi BS_AN chạy SELECT *, số dòng vẫn được giữ nguyên, HOTEN có giá trị, còn các cột bị che trả về NULL. Cách này đáp ứng yêu cầu “SELECT mức cột” mà không sửa cấu trúc bảng và không yêu cầu người dùng đổi câu truy vấn.

1.4.4. SELECT mức cột trên VIEW bằng view trung gian

Đối với view, nhóm không gắn VPD theo từng grantee mà tạo một view trung gian chỉ chứa các cột được phép. Tên view được sinh từ tên view gốc và danh sách cột đã chuẩn hóa, ví dụ V_V_DANH_SACH_NHAN_VIEN_USERNAME_HOTEN. Nếu một tổ hợp cột đã tồn tại, hệ thống tái sử dụng view đó; nếu chưa có, procedure tạo mới rồi cấp SELECT trên view trung gian.

Cách làm này tạo ra một interface dữ liệu rõ ràng: user không có quyền trên view gốc mà chỉ có quyền trên projection đã được giới hạn. Điểm đổi lại là số lượng view có thể tăng khi có nhiều tổ hợp cột khác nhau; vì vậy, việc chuẩn hóa thứ tự cột và tái sử dụng view là cần thiết để tránh tạo trùng.

1.4.5. INSERT, DELETE, UPDATE và cấp quyền trên procedure/function

INSERT và DELETE được giữ ở mức toàn table/view theo đúng phạm vi đề bài, còn procedure và function chỉ hỗ trợ EXECUTE. Các lệnh cấp quyền này bao gồm cả UPDATE toàn table được Oracle hỗ trợ trực tiếp.

Ngoài ra, Oracle cũng hỗ trợ trực tiếp UPDATE mức cột. SP_GRANT_PRIVILEGE xây dựng câu GRANT UPDATE(col1, col2) ON object TO grantee. Khi đó user chỉ có thể cập nhật các cột đã được liệt kê; thao tác UPDATE trên cột khác sẽ bị Oracle từ chối.

1.5. Yêu cầu 4 - Thu hồi quyền

1.5.1. Cơ sở lý thuyết

REVOKE loại bỏ quyền hoặc role đã được cấp trực tiếp. Khi một object privilege bị thu hồi từ user, các quyền phụ thuộc mà user đó đã cấp tiếp cũng bị thu hồi. Tuy nhiên, nếu user vẫn nhận cùng quyền từ một grantor khác hoặc thông qua role, quyền hiệu lực có thể vẫn còn. Vì vậy, việc đánh giá kết quả thu hồi cần xem cả nguồn cấp trực tiếp và gián tiếp.

Oracle không hỗ trợ cú pháp REVOKE UPDATE(column). Để thu hồi một phần quyền UPDATE mức cột, cần thu hồi UPDATE trên toàn đối tượng rồi cấp lại các cột còn được phép. Tương tự, SELECT mức cột của đồ án không phải quyền native nên việc thu hồi phải đồng bộ thêm VPD_COL_TRACKING, VPD policy hoặc view trung gian.

1.5.2. Giải pháp thực hiện

Việc thu hồi được thực hiện theo từng trường hợp:

Trường hợp	Cách xử lý trong SP_REVOKE_PRIVILEGE
SELECT một số cột trên TABLE	Bổ sung các cột bị thu hồi vào HIDDEN_COLS, tạo lại VPD policy; nếu mọi cột đều bị ẩn thì xóa policy, xóa tracking và REVOKE SELECT toàn bảng.
SELECT toàn TABLE	Xóa VPD policy nếu có, REVOKE SELECT và xóa bản ghi VPD_COL_TRACKING.
SELECT một số cột trên VIEW	Tìm view trung gian hiện tại, tính các cột còn lại, tạo/tái sử dụng view mới, cấp view mới và thu hồi view cũ.
SELECT toàn VIEW	Thu hồi quyền trên view gốc và toàn bộ view trung gian có tiền tố tương ứng.
UPDATE một số cột	Đọc các cột còn lại từ DBA_COL_PRIVS, REVOKE UPDATE toàn đối tượng, sau đó GRANT UPDATE lại cho các cột còn lại.

Trường hợp	Cách xử lý trong SP_REVOKE_PRIVILEGE
INSERT, DELETE, EXECUTE, ROLE, SYSTEM	Thực hiện REVOKE trực tiếp theo loại quyền hoặc role.

Đối với VPD, thao tác thu hồi không chỉ thay đổi quyền SELECT mà còn thay đổi danh sách cột bị che. Procedure đếm tổng số cột và số cột bị ẩn; nếu số cột bị ẩn bằng tổng số cột thì SELECT được thu hồi hoàn toàn. Cách xử lý này giữ cho trạng thái quyền trong Data Dictionary và trạng thái policy trong VPD_COL_TRACKING nhất quán.

1.5.3. Kết quả đạt được

Hệ thống thu hồi được quyền ở mức toàn đối tượng, role và mức cột. Điểm quan trọng là các quyền mô phỏng bằng VPD hoặc view trung gian được thu hồi theo đúng cơ chế đã dùng khi cấp, thay vì chỉ gọi REVOKE và để lại policy hoặc view không còn phù hợp.

1.6. Yêu cầu 5 - Tra cứu quyền

1.6.1. Cơ sở lý thuyết

Thông tin quyền được lấy từ các view Data Dictionary. DBA_TAB_PRIVS lưu object privilege ở mức đối tượng; DBA_COL_PRIVS lưu quyền mức cột native; DBA_SYS_PRIVS lưu system privilege; DBA_ROLE_PRIVS lưu các role được cấp; DBA_POLICIES cho biết các VPD policy đã đăng ký. Các view này là nguồn dữ liệu đáng tin cậy để DBA kiểm tra trạng thái phân quyền.

- Danh sách các Data Dictionary view liên quan:

View	Thông tin khai thác
DBA_USERS	User, trạng thái tài khoản, tablespace mặc định, ngày tạo, cờ ORACLE_MAINTAINED.
DBA_ROLES	Danh sách role và kiểu xác thực.
DBA_ROLE_PRIVS	Role đã cấp cho user/role, ADMIN_OPTION, DEFAULT_ROLE.
DBA_TAB_PRIVS	Object privilege ở mức table/view/procedure/function và GRANTABLE.
DBA_COL_PRIVS	Quyền mức cột native, chủ yếu UPDATE trong giải pháp này.
DBA_SYS_PRIVS	System privilege và ADMIN_OPTION.
DBA_TAB_COLUMNS	Danh sách và thứ tự cột để tính cột được phép/cột bị ẩn.
DBA_POLICIES / DBA_VIEWS	Kiểm tra VPD policy và view trung gian.

1.6.2. Giải pháp thực hiện

USP_GET_OBJ_PRIVS tổng hợp ba nguồn bằng UNION ALL. Nhánh thứ nhất lấy quyền ở mức toàn đối tượng từ DBA_TAB_PRIVS. Nhánh thứ hai kết hợp DBA_TAB_PRIVS, VPD_COL_TRACKING và DBA_TAB_COLUMNS để chuyển quyền SELECT dùng VPD thành từng dòng cột được phép. Nhánh thứ ba lấy quyền UPDATE mức cột trực tiếp từ DBA_COL_PRIVS. Nhờ vậy, giao diện có thể hiển thị quyền theo cùng một cấu trúc dù cơ chế lưu trữ bên dưới khác nhau.

USP_GET_ROLE_PRIVS trả các role đã cấp cho một grantee cùng ADMIN_OPTION và DEFAULT_ROLE. Bên cạnh màn hình tra cứu, script còn có USP_DASHBOARD_STATS, USP_DASHBOARD_ROLE_DISTRIBUTION và USP_DASHBOARD_OBJECTS để tổng hợp số lượng user, role, quyền và object, cũng như phân bổ user theo role.

1.6.3. Kết quả đạt được

DBA có thể kiểm tra trực tiếp quyền của user hoặc role mà không cần truy vấn thủ công nhiều view hệ thống. Đặc biệt, màn hình vẫn biểu diễn được quyền SELECT mức cột do nhóm tự xây dựng bằng VPD, thay vì chỉ hiển thị SELECT toàn table như DBA_TAB_PRIVS. Đây là thành phần giúp quá trình cấp và thu hồi quyền có thể được kiểm tra lại ngay trên ứng dụng.

1.7. Đánh giá phân hệ

Phân hệ 1 đã xây dựng một công cụ quản trị Oracle có giao diện, đáp ứng các yêu cầu chính về user, role và quyền truy cập. Hệ thống sử dụng stored procedure để thực thi DDL/DCL, Data Dictionary để truy xuất metadata, VPD để che cột trên table, view trung gian để giới hạn cột trên view và quyền UPDATE mức cột native của Oracle. Các chức năng cấp, thu hồi và tra cứu được liên kết thành một quy trình thống nhất trên ứng dụng WinForms.

1.7.1. Ưu điểm

- **Bao phủ đầy đủ yêu cầu chức năng:** Hệ thống xử lý quản lý user/role, cấp quyền, thu hồi quyền và tra cứu quyền trên cả bốn loại đối tượng của đề bài.
- **Tập trung logic tại CSDL:** DDL/DCL được đóng gói trong procedure, giảm trùng lặp ở tầng C# và giúp các màn hình sử dụng cùng một quy tắc xử lý.
- **Giải quyết được SELECT mức cột:** VPD column masking được dùng cho table, còn view trung gian được dùng cho view; đây là phần mở rộng cần thiết vì Oracle không hỗ trợ GRANT SELECT(column).
- **Có khả năng kiểm tra ngược:** Data Dictionary và VPD_COL_TRACKING được tổng hợp để giao diện hiển thị cả quyền native lẫn quyền do nhóm mô phỏng.

- **Tích hợp giao diện C# Winforms với Oracle:** Theo cấu trúc dự án nhóm cung cấp, phần giao diện Phân hệ 1 nằm trong thư mục GUI/PH1 và gồm các màn hình Dashboard, User, Role, Grant/Revoke và View Privilege. Lớp DAO/DataProvider.cs sử dụng Oracle.ManagedDataAccess để quản lý kết nối và thực thi procedure. Luồng xử lý chung là: người dùng thao tác trên control → giao diện kiểm tra dữ liệu đầu vào → DataProvider tạo OracleCommand → gọi stored procedure → nhận SYS_REFCURSOR hoặc số dòng ảnh hưởng → cập nhật giao diện và thông báo kết quả.

1.7.2. Hạn chế

- Tên user, role, object và tablespace được nối trực tiếp vào dynamic SQL.
 - + Ảnh hưởng: Nếu dữ liệu đầu vào không được kiểm soát chặt, có nguy cơ SQL injection qua identifier.
 - + Hướng khắc phục: Xác thực allow-list và sử dụng DBMS_ASSERT.ENQUOTE_NAME/SQL_OBJECT_NAME phù hợp; không chỉ dựa vào kiểm tra ở giao diện.
- Tên view trung gian được sinh từ tổ hợp cột và cắt ở 128 ký tự.
 - + Ảnh hưởng: Có thể phát sinh tên dài hoặc va chạm khi view có nhiều cột.
 - + Hướng khắc phục: Sinh tên bằng mã băm ổn định và lưu mapping trong bảng metadata.
- Một số thông kê dashboard chủ yếu phản ánh quyền trực tiếp.
 - + Ảnh hưởng: Có thể chưa thể hiện đầy đủ quyền hiệu lực qua role lồng nhau.
 - + Hướng khắc phục: Bổ sung truy vấn phân cấp role hoặc một màn hình “effective privileges”.

2. Phân hệ 2 - Ứng dụng quản lý dữ liệu y tế

2.1. Giới thiệu phân hệ

Phân hệ 2 xây dựng ứng dụng quản lý dữ liệu y tế cho bệnh viện X, tập trung vào việc quản lý và bảo vệ các thông tin nhạy cảm liên quan đến bệnh nhân, nhân viên y tế và hoạt động khám chữa bệnh. Khác với các hệ thống quản lý dữ liệu thông thường, dữ liệu y tế yêu cầu mức độ bảo mật cao do chứa nhiều thông tin cá nhân, hồ sơ bệnh án, kết quả xét nghiệm và các thông tin chỉ được phép truy cập bởi những đối tượng có thẩm quyền.

Hệ thống được xây dựng trên nền tảng Oracle Database, kết hợp các cơ chế bảo mật tích hợp như kiểm soát truy cập dựa trên vai trò (Role-Based Access Control - RBAC), kiểm soát truy cập chi tiết bằng Virtual Private Database (VPD), Oracle Label Security (OLS), Audit và các cơ chế sao lưu phục hồi dữ liệu. Mục tiêu của phân hệ là đảm bảo mỗi nhóm người dùng chỉ có thể truy cập đúng phạm vi dữ liệu cần thiết theo nhiệm vụ được giao, đồng thời duy trì khả năng giám sát và phục hồi khi xảy ra sự cố.

Trong hệ thống, các đối tượng sử dụng được phân chia thành các nhóm chính:

- Bệnh nhân: có quyền xem các thông tin thuộc về chính bản thân như hồ sơ cá nhân, lịch sử khám bệnh hoặc các thông tin được bệnh viện cho phép cung cấp. Bệnh nhân không được phép truy cập dữ liệu của các bệnh nhân khác.
- Kỹ thuật viên: thực hiện các nghiệp vụ liên quan đến xét nghiệm, chẩn đoán hình ảnh hoặc cập nhật kết quả dịch vụ y tế. Người dùng thuộc nhóm này chỉ được phép thao tác trên những dữ liệu phục vụ cho công việc chuyên môn.
- Điều phối viên: chịu trách nhiệm quản lý và điều phối hoạt động khám chữa bệnh. Tuy nhiên, người dùng thuộc vai trò này không cần truy cập toàn bộ dữ liệu y tế mà chỉ được xem các thông tin cần thiết phục vụ công tác điều phối.
- Bác sĩ: là nhóm người dùng trực tiếp làm việc với hồ sơ bệnh án. Do đặc thù công việc, bác sĩ cần truy cập dữ liệu bệnh nhân nhưng phạm vi truy cập phải được giới hạn theo bệnh nhân hoặc khoa phụ trách nhằm đảm bảo tính riêng tư của thông tin y tế.

Bên cạnh việc kiểm soát quyền truy cập dữ liệu, hệ thống còn yêu cầu xây dựng cơ chế phát tán thông báo nội bộ sử dụng Oracle Label Security. Các thông báo được gán nhãn bảo mật dựa trên ba thuộc tính gồm cấp độ truy cập (Level), phạm vi chuyên môn (Compartment) và khu vực làm việc (Group). Nhờ đó, người dùng chỉ có thể đọc các thông báo phù hợp với nhãn bảo mật đã được cấp.

Ngoài ra, hệ thống cần ghi nhận các hoạt động quan trọng xảy ra trên cơ sở dữ liệu thông qua cơ chế Audit. Việc kiểm toán giúp DBA theo dõi các thao tác truy cập, chỉnh sửa dữ liệu, phát hiện các hành vi bất thường và hỗ trợ truy vết khi cần thiết. Đối với dữ liệu quan

trọng, hệ thống cũng cần triển khai các phương pháp sao lưu và phục hồi nhằm đảm bảo khả năng khôi phục dữ liệu khi xảy ra lỗi hoặc sự cố ngoài mong muốn.

Từ các yêu cầu trên, Phân hệ 2 tập trung giải quyết bốn nhóm vấn đề chính:

- Thiết lập chính sách cấp quyền truy cập dữ liệu
 - + Áp dụng RBAC cho các nhóm người dùng có quyền truy cập tương tự.
 - + Áp dụng VPD khi cần giới hạn dữ liệu theo điều kiện cụ thể.
- Bảo vệ thông tin thông báo bằng Oracle Label Security
 - + Xây dựng mô hình nhãn bảo mật.
 - + Kiểm soát khả năng đọc thông báo theo quyền của từng người dùng.
- Giám sát hoạt động trên hệ thống thông qua Audit
 - + Theo dõi các thao tác truy cập và thay đổi dữ liệu.
 - + Lưu trữ nhật ký phục vụ kiểm tra và đánh giá bảo mật.
- Đảm bảo an toàn dữ liệu thông qua sao lưu và phục hồi
 - + Thử nghiệm các phương pháp backup/recovery trên Oracle Database.
 - + Đánh giá khả năng khôi phục dữ liệu sau sự cố.

Thông qua các cơ chế bảo mật trên, hệ thống hướng đến mục tiêu đảm bảo ba yếu tố quan trọng của an toàn thông tin gồm tính bảo mật (Confidentiality), tính toàn vẹn (Integrity) và tính sẵn sàng (Availability) của dữ liệu y tế.

2.2. Yêu cầu 1 - Giải pháp cấp quyền truy cập

2.2.1. Thiết lập tài khoản người dùng

a. Lý thuyết

Authentication là quá trình kiểm tra và xác minh danh tính của người dùng trước khi cho phép truy cập vào hệ thống. Trong Oracle Database, việc xác thực thường được thực hiện thông qua username và password được lưu trữ và quản lý bởi hệ quản trị cơ sở dữ liệu. Khi người dùng đăng nhập thành công, Oracle sẽ xác định danh tính và thiết lập một phiên làm việc tương ứng. Từ đó, các cơ chế phân quyền, kiểm soát truy cập và kiểm toán sẽ được áp dụng dựa trên tài khoản đang đăng nhập.

Oracle User là đối tượng đại diện cho một tài khoản đăng nhập trong hệ quản trị cơ sở dữ liệu Oracle. Mỗi user bao gồm tên đăng nhập, mật khẩu, quyền hệ thống, quyền trên đối tượng dữ liệu và các Role được gán. Mọi thao tác truy cập dữ liệu đều được Oracle ghi nhận theo tài khoản người dùng đang thực hiện.

b. Giải pháp thực hiện

Hệ thống được triển khai trên Oracle Database với mô hình quản lý tập trung thông qua schema nghiệp vụ ADMINHOS. Trong quá trình cài đặt, DBA (SYS) thực hiện tạo Pluggable Database (PDBHOSX) và tạo tài khoản ADMINHOS làm chủ sở hữu dữ liệu của hệ thống bệnh viện. Toàn bộ các đối tượng dữ liệu và cơ chế bảo mật đều được xây dựng trong schema này.

ADMINHOS chịu trách nhiệm:

1. Tạo và quản lý các bảng dữ liệu nghiệp vụ:
 - BENHNHAN
 - NHANVIEN
 - HSBA
 - HSBA_DV
 - DONTUOC
2. Tạo và quản lý các đối tượng hỗ trợ:
 - View
 - Stored Procedure
 - Function
 - Trigger
3. Triển khai các cơ chế bảo mật:
 - Role-Based Access Control (RBAC)
 - Virtual Private Database (VPD)
 - Audit và Fine-Grained Audit (FGA)
4. Cấp quyền cho các tài khoản người dùng trong hệ thống.

Việc sử dụng một schema trung tâm giúp việc quản lý dữ liệu, bảo mật và kiểm toán được thực hiện thống nhất, đồng thời giảm sự phân tán dữ liệu trong hệ thống.

Theo yêu cầu TC#1, hệ thống không được xây dựng bảng quản lý tài khoản riêng mà phải sử dụng trực tiếp cơ chế quản lý người dùng của Oracle. Để đáp ứng yêu cầu này, nhóm thiết kế tài khoản Oracle trùng với mã định danh của người dùng trong dữ liệu nghiệp vụ.

Đối tượng	Username oracle
Điều phối viên	DP0001
Bác sĩ/Y sĩ	BS0001
Kỹ thuật viên	KTV001
Bệnh nhân	BN000001

Trong cơ sở dữ liệu:

- Mã nhân viên (MANV) được đặt trùng với username Oracle đối với nhân viên.
- Mã bệnh nhân (MABN) được đặt trùng với username Oracle đối với bệnh nhân.

Username	Dữ liệu liên kết
BS0001	MANV = BS0001
DP0001	MANV = DP0001
KTV001	MANV = KTV001
BN000001	MABN = BN000001

Khi người dùng đăng nhập vào hệ thống, Oracle xác định tài khoản hiện hành thông qua hàm: `SYS_CONTEXT('USERENV', 'SESSION_USER')`.

Giá trị trả về chính là username Oracle của người dùng đang đăng nhập. Từ đó hệ thống có thể xác định chính xác bản ghi tương ứng trong bảng NHANVIEN hoặc BENHNNHAN mà không cần sử dụng thêm bất kỳ bảng ánh xạ tài khoản nào.

Giải pháp này đáp ứng yêu cầu của đề bài về việc liên kết tài khoản Oracle với dữ liệu người dùng chỉ thông qua một nguồn dữ liệu duy nhất, đồng thời tạo nền tảng cho việc xây dựng các chính sách RBAC, VPD và Audit ở các phần tiếp theo.

c. Kết quả đạt được

Xây dựng thành công schema nghiệp vụ ADMINHOS làm trung tâm quản lý toàn bộ dữ liệu bệnh viện. Mỗi nhân viên và bệnh nhân được cấp một tài khoản Oracle riêng biệt. Tài khoản Oracle được liên kết trực tiếp với dữ liệu nghiệp vụ thông qua MANV hoặc MABN. Không cần xây dựng bảng quản lý tài khoản trung gian. Dễ dàng xác định người dùng hiện hành thông qua `SYS_CONTEXT`. Hỗ trợ hiệu quả cho việc triển khai các cơ chế RBAC, VPD và Audit. Đáp ứng đầy đủ yêu cầu TC#1 của đề bài về quản lý tài khoản và xác thực người dùng.

2.2.2. RBAC cho Bệnh nhân và Kỹ thuật viên

a. Lý thuyết

RBAC (Role-Based Access Control) là mô hình kiểm soát truy cập dựa trên vai trò, trong đó quyền truy cập không được cấp trực tiếp cho từng người dùng mà được gán cho các vai trò. Người dùng sau đó được gán vào một hoặc nhiều vai trò tương ứng với chức năng công việc của mình. Mô hình này giúp đơn giản hóa việc quản lý quyền, giảm đáng kể số lượng lệnh cấp quyền cần thực hiện, đồng thời dễ dàng mở rộng khi số lượng người dùng tăng lên và đảm bảo nguyên tắc phân quyền tối thiểu.

Trong Oracle, Role là tập hợp các quyền hệ thống và quyền trên đối tượng dữ liệu được gom nhóm lại thành một đơn vị quản lý. Oracle cho phép tạo Role, gán quyền cho Role và gán Role cho nhiều người dùng khác nhau.

Ví dụ, nhóm tạo hai Role là `ROLE_BENHNHAN` và `ROLE_KTV`, sau đó các quyền được cấp cho Role thay vì cấp trực tiếp cho từng tài khoản.

b. Nhận xét

Trong hệ thống bệnh viện, số lượng người dùng là rất lớn với khoảng 100.000 bệnh nhân và 50 kỹ thuật viên. Tuy nhiên, các đối tượng thuộc cùng một nhóm vai trò lại có tập quyền truy cập tương đối giống nhau. Nếu thực hiện cấp quyền trực tiếp cho từng tài khoản, hệ thống sẽ trở nên khó quản lý, khó mở rộng và dễ xảy ra sai sót mỗi khi cần cập nhật hoặc thay đổi quyền.

Xuất phát từ đặc điểm trên, nhóm lựa chọn cơ chế RBAC để quản lý quyền cho hai nhóm người dùng này. RBAC là giải pháp phù hợp bởi các bệnh nhân có cùng nhu cầu truy cập dữ liệu cá nhân, các kỹ thuật viên có cùng phạm vi chức năng nghiệp vụ, và việc cấp hoặc thu hồi quyền có thể được thực hiện tập trung thông qua Role thay vì thao tác trên từng tài khoản riêng lẻ.

c. Giải pháp thực hiện

Nhóm xây dựng hai Role chính là `ROLE_BENHNHAN` và `ROLE_KTV`, mỗi Role đại diện cho tập quyền của một nhóm người dùng tương ứng. Tài khoản người dùng sau khi được tạo sẽ được gán Role phù hợp, ví dụ `GRANT ROLE_BENHNHAN TO BN000001`.

Đối với bệnh nhân, `ROLE_BENHNHAN` cho phép người dùng xem thông tin cá nhân của chính mình và cập nhật một số trường được phép thay đổi. Cụ thể:

- Các trường được phép cập nhật: `SONHA`, `TENDUONG`, `QUANHUYEN`, `TINHTP`, `TIENSUBENH`, `TIENSUBENHGD` và `DIUNGTHUOC`.
- Các trường không được phép cập nhật: `MABN`, `TENBN`, `PHAI`, `NGAYSINH` và `CCCD`.

Để thực hiện yêu cầu TC#5, hệ thống sử dụng View kết hợp với cơ chế bảo mật nhằm đảm bảo mỗi bệnh nhân chỉ nhìn thấy dữ liệu của chính mình mà không thể truy cập thông tin của bệnh nhân khác.

Đối với kỹ thuật viên, `ROLE_KTV` cho phép:

- Xem thông tin các dịch vụ hỗ trợ chẩn đoán được phân công.
- Cập nhật trường `KETQUA` trong bảng `HSBA_DV`.

Theo yêu cầu TC#4, mỗi kỹ thuật viên chỉ được thao tác trên các dịch vụ do chính mình thực hiện, ví dụ `KTV001` chỉ xem các dòng có `MAKTV = KTV001`. Ngoài phạm vi trên,

kỹ thuật viên không được phép xem toàn bộ hồ sơ bệnh án, cập nhật chẩn đoán, điều trị, kết luận, quản lý đơn thuốc hay chỉnh sửa thông tin bệnh nhân. Việc giới hạn phạm vi dữ liệu này được thực hiện bằng cách kết hợp giữa RBAC và các chính sách bảo mật mức dòng dữ liệu.

Sau khi tạo tài khoản Oracle cho người dùng, ADMINHOS thực hiện gán Role tương ứng. Người dùng sau khi đăng nhập sẽ kế thừa toàn bộ quyền được định nghĩa trong Role, nhờ đó việc quản lý hàng chục nghìn tài khoản bệnh nhân và hàng chục kỹ thuật viên được thực hiện tập trung và hiệu quả.

d. Kết quả đạt được

Đối với bệnh nhân, hệ thống đảm bảo mỗi người chỉ xem được thông tin cá nhân của chính mình và chỉ chỉnh sửa được các trường được cho phép, không thể truy cập hay thay đổi dữ liệu của bệnh nhân khác. Kết quả này đáp ứng đầy đủ yêu cầu TC#5 của đề bài.

Đối với kỹ thuật viên, mỗi người chỉ xem được các dịch vụ được phân công cho mình và chỉ được cập nhật trường KETQUA, không được truy cập các dữ liệu điều trị ngoài phạm vi nhiệm vụ. Kết quả này đáp ứng đầy đủ yêu cầu TC#4 của đề bài.

Nhìn chung, việc áp dụng RBAC đã giúp giảm đáng kể số lượng lệnh cấp quyền trực tiếp, cho phép quản lý tập trung thông qua Role và dễ dàng mở rộng khi số lượng người dùng tăng. Đồng thời, cơ chế này đảm bảo nguyên tắc quyền tối thiểu và tăng tính an toàn cũng như khả năng quản trị cho toàn bộ hệ thống.

2.2.3. VPD cho Điều phối viên và Bác sĩ

a. Lý thuyết

Virtual Private Database (VPD) là cơ chế bảo mật mức dòng dữ liệu (Row-Level Security) được Oracle cung cấp thông qua gói DBMS_RLS. VPD cho phép hệ quản trị cơ sở dữ liệu tự động bổ sung điều kiện WHERE vào các câu lệnh SQL của người dùng khi truy cập dữ liệu. Người dùng không nhìn thấy điều kiện này, nhưng Oracle sẽ tự động áp dụng trước khi thực thi truy vấn, nhờ đó người dùng chỉ nhìn thấy những dữ liệu mà mình được phép truy cập.

Row-Level Security là cơ chế kiểm soát truy cập đến từng dòng dữ liệu thay vì toàn bộ bảng. Khác với RBAC chỉ kiểm soát các thao tác SELECT, INSERT, UPDATE, DELETE trên toàn bộ bảng hoặc cột, VPD kiểm soát chi tiết hơn ở mức từng dòng: dòng nào được xem, dòng nào được cập nhật, dòng nào được xóa và dòng nào được thêm mới. Điều này đặc biệt phù hợp với hệ thống bệnh viện, nơi mỗi bác sĩ phụ trách các bệnh nhân khác nhau và chỉ nên truy cập dữ liệu trong phạm vi điều trị của mình.

b. Nhận xét

RBAC là cơ chế phân quyền hiệu quả theo vai trò, tuy nhiên chỉ kiểm soát được quyền thao tác trên đối tượng dữ liệu mà không thể giới hạn phạm vi dữ liệu cụ thể của từng người dùng.

Trong hệ thống bệnh viện, nhiều người dùng cùng một vai trò nhưng được phép truy cập các tập dữ liệu khác nhau. Chẳng hạn, mọi bác sĩ đều cần quyền xem và cập nhật bảng HSBA, nhưng chỉ được thao tác trên các hồ sơ bệnh án do mình phụ trách. Nếu chỉ sử dụng RBAC, bác sĩ có thể truy cập toàn bộ dữ liệu của bảng HSBA, không đáp ứng yêu cầu bảo mật của đề bài.

Vì vậy, nhóm kết hợp RBAC với cơ chế Virtual Private Database (VPD). RBAC được sử dụng để quản lý quyền truy cập theo vai trò, trong khi VPD thực hiện kiểm soát ở mức dòng dữ liệu, đảm bảo mỗi người dùng chỉ được truy cập các thông tin thuộc phạm vi trách nhiệm của mình. Điều này giúp hệ thống đáp ứng đầy đủ các yêu cầu bảo mật đối với vai trò Điều phối viên và Bác sĩ/Y sĩ.

c. Giải pháp thực hiện

Trong Oracle, VPD hoạt động dựa trên các Policy Function. Mỗi Policy Function có nhiệm vụ xác định người dùng hiện tại, sinh ra điều kiện lọc dữ liệu phù hợp và trả về một Predicate để Oracle tự động gắn vào câu lệnh SQL trước khi thực thi. Người dùng hiện tại được xác định thông qua hàm `SYS_CONTEXT('USERENV', 'SESSION_USER')`. Ví dụ, khi tài khoản BS0001 đăng nhập, hàm này sẽ trả về giá trị BS0001, từ đó Policy Function sử dụng giá trị này để sinh ra điều kiện lọc tương ứng.

Nhóm triển khai cơ chế VPD trên schema ADMINHOS thông qua package `PKG_VPD_YC1C3` và các policy được tạo bằng thủ tục `DBMS_RLS.ADD_POLICY`.

Đầu tiên, package `PKG_VPD_YC1C3` xây dựng các hàm kiểm tra vai trò người dùng đăng nhập dựa trên giá trị `SYS_CONTEXT('USERENV','SESSION_USER')`. Hai hàm `IS_DPV` và `IS_BS` được sử dụng để xác định tài khoản hiện tại là Điều phối viên hay Bác sĩ/Y sĩ thông qua dữ liệu trong bảng `NHANVIEN`.

Đối với vai trò **Điều phối viên**, nhóm xây dựng các hàm:

- `FN_BENHNHAN_DPV`
- `FN_HSBA_SEL_DPV`
- `FN_HSBA_UPD_DPV`
- `FN_HSBADV_DPV`

Các hàm này được gắn vào các bảng BENHNHAN, HSBA và HSBA_DV nhằm cho phép Điều phối viên thực hiện đúng các nghiệp vụ theo yêu cầu đề bài như quản lý bệnh nhân, tạo hồ sơ bệnh án, phân công bác sĩ và phân công kỹ thuật viên.

Ngoài ra nhóm còn xây dựng view VW_NHANVIEN_DIEUPHOI để cung cấp danh sách nhân viên phục vụ công tác điều phối. View này chỉ hiển thị các nhân viên có vai trò **Bác sĩ/Y sĩ** hoặc **Kỹ thuật viên**. Đồng thời, dữ liệu được lọc theo cơ sở làm việc (COSO) của điều phối viên đang đăng nhập. Nhờ đó, điều phối viên chỉ nhìn thấy các nhân sự thuộc phạm vi cơ sở mình phụ trách để thực hiện việc phân công.

Đối với vai trò **Bác sĩ/Y sĩ**, nhóm xây dựng các hàm:

- FN_HSBA_BS
- FN_HSBADV_BS
- FN_DONTHUOC_BS
- FN_BENHNHAN_BS

Các hàm này trả về điều kiện lọc dữ liệu dựa trên mã bác sĩ (MABS) hoặc các hồ sơ bệnh án liên quan đến bác sĩ đang đăng nhập. Nhờ đó, bác sĩ chỉ có thể xem và thao tác trên các hồ sơ bệnh án, dịch vụ hỗ trợ chẩn đoán, đơn thuốc và bệnh nhân thuộc phạm vi điều trị của mình.

Sau khi xây dựng các policy function, nhóm sử dụng DBMS_RLS.ADD_POLICY để gắn các chính sách VPD vào bốn bảng nghiệp vụ chính gồm BENHNHAN, HSBA, HSBA_DV và DONTHUOC. Các policy được áp dụng cho các thao tác SELECT, INSERT, UPDATE hoặc DELETE tùy theo yêu cầu của từng vai trò.

Bên cạnh đó, nhóm sử dụng tài khoản ADMINHOS làm chủ sở hữu dữ liệu và quản lý toàn bộ policy VPD. Các quyền truy cập đối tượng được cấp trực tiếp thông qua hai thủ tục SP_YC1C3_GRANT_DPV và SP_YC1C3_GRANT_BS, trong khi phạm vi dữ liệu được kiểm soát bởi VPD. Cách triển khai này giúp kết hợp giữa cấp quyền ở mức đối tượng và kiểm soát dữ liệu ở mức dòng, đáp ứng đầy đủ yêu cầu TC#2 và TC#3 của đề bài.

d. Kết quả đạt được

Sau khi triển khai cơ chế VPD kết hợp với các view hỗ trợ, hệ thống đã đáp ứng đầy đủ các yêu cầu bảo mật đối với hai nhóm người dùng Điều phối viên và Bác sĩ/Y sĩ theo TC#2 và TC#3.

Đối với Điều phối viên, các policy VPD được áp dụng trên các bảng nghiệp vụ như BENHNHAN, HSBA và HSBA_DV, giúp giới hạn dữ liệu theo đúng phạm vi quản lý của người dùng. Khi đăng nhập, điều phối viên chỉ có thể truy cập các dữ liệu phục vụ cho công tác tiếp nhận bệnh nhân, tạo hồ sơ bệnh án và phân công nhân sự. Bên cạnh đó, view

VW_NHANVIEN_DIEUPHOI cho phép điều phối viên chỉ xem danh sách bác sĩ/y sĩ và kỹ thuật viên thuộc cùng cơ sở làm việc để thực hiện việc phân công, tránh truy cập đến các nhân sự không liên quan.

Đối với Bác sĩ/Y sĩ, các policy VPD được áp dụng trên các bảng HSBA, HSBA_DV, DONTUOC và BENHNHAN. Kết quả là mỗi bác sĩ chỉ có thể xem và thao tác trên các hồ sơ bệnh án mà mình phụ trách điều trị. Từ các hồ sơ này, bác sĩ được phép quản lý đơn thuốc, dịch vụ hỗ trợ chẩn đoán và cập nhật thông tin bệnh nhân theo đúng yêu cầu nghiệp vụ của đề bài.

Kết quả kiểm thử cho thấy các tài khoản có cùng quyền chức năng nhưng khác danh tính đăng nhập sẽ nhận được các tập dữ liệu khác nhau. Oracle tự động áp dụng các điều kiện lọc do VPD sinh ra mà không cần xử lý ở tầng ứng dụng. Điều này đảm bảo dữ liệu y tế được bảo vệ ở mức dòng dữ liệu, đồng thời đáp ứng đầy đủ các yêu cầu phân quyền và bảo mật được đặt ra trong đề án.

2.3. Yêu cầu 2 - Oracle Label Security (OLS)

2.3.1. Cơ sở lý thuyết

a. Mandatory Access Control (MAC)

Mandatory Access Control (MAC) là mô hình kiểm soát truy cập bắt buộc, trong đó quyền truy cập dữ liệu không do chủ sở hữu đối tượng quyết định mà được quy định bởi hệ thống thông qua các nhãn bảo mật. Mỗi đối tượng dữ liệu và mỗi người dùng đều được gán một nhãn, và hệ thống sẽ tự động so sánh nhãn của người dùng với nhãn của dữ liệu để quyết định cho phép hay từ chối truy cập. Khác với mô hình DAC cho phép chủ sở hữu tự do cấp quyền, MAC đảm bảo tính bảo mật nghiêm ngặt hơn vì ngay cả chủ sở hữu dữ liệu cũng không thể vượt qua chính sách nhãn do hệ thống áp đặt.

b. Oracle Label Security (OLS)

Oracle Label Security (OLS) là hiện thực của mô hình MAC trên nền tảng Oracle Database. OLS cho phép gán nhãn bảo mật lên từng dòng dữ liệu trong bảng và gán nhãn truy cập cho từng người dùng. Khi người dùng thực hiện truy vấn, Oracle sẽ tự động so sánh nhãn của người dùng với nhãn của từng dòng dữ liệu và chỉ trả về những dòng mà người dùng có đủ quyền đọc. Cơ chế này hoạt động trong suốt đối với ứng dụng, nghĩa là câu lệnh SQL không cần thay đổi nhưng kết quả trả về sẽ khác nhau tùy theo nhãn của người đang đăng nhập.

Mỗi nhãn bảo mật trong OLS được cấu thành từ ba thành phần:

- **Level (cấp độ):** xác định mức độ nhạy cảm của thông tin theo thứ tự phân cấp, level cao hơn có thể đọc được dữ liệu của level thấp hơn.
- **Compartment (phạm vi chuyên môn):** phân loại dữ liệu theo lĩnh vực hoặc chuyên khoa, người dùng chỉ đọc được dữ liệu thuộc compartment mà mình được gán.
- **Group (nhóm khu vực):** phân loại dữ liệu theo đơn vị tổ chức hoặc vị trí địa lý, giới hạn phạm vi truy cập theo khu vực làm việc.

Cú pháp biểu diễn nhãn có dạng: **LEVEL:COMPARTMENT:GROUP**. Ví dụ, nhãn **LDK:TH:HCM** nghĩa là cấp Lãnh đạo khoa, thuộc Khoa tiêu hóa, tại khu vực Hồ Chí Minh.

2.3.2. Thiết kế nhãn bảo mật

Dựa trên cơ cấu tổ chức của bệnh viện, nhóm thiết kế mô hình nhãn bảo mật gồm ba thành phần như sau:

Level - Cấp độ truy cập

Hệ thống định nghĩa ba cấp độ truy cập theo thứ tự phân cấp từ thấp đến cao. Cấp Nhân viên (NV, mức 10) là cấp thấp nhất, dành cho toàn bộ nhân viên trong bệnh viện. Cấp Lãnh đạo khoa (LDK, mức 20) dành cho các trưởng/phó khoa, có thể đọc được cả thông báo cấp Nhân viên. Cấp Ban giám đốc (BGD, mức 30) là cấp cao nhất, có thể đọc được toàn bộ thông báo của các cấp bên dưới.

Mức	Viết tắt	Mô tả
30	BGD	Ban giám đốc
20	LDK	Lãnh đạo khoa
10	NV	Nhân viên

Compartment - Phạm vi chuyên khoa

Thông báo được phân loại theo ba chuyên khoa chính của bệnh viện: Khoa tiêu hóa (TH), Khoa thần kinh (TK) và Khoa tim mạch (TM). Người dùng chỉ đọc được thông báo thuộc chuyên khoa mà mình được gán. Riêng đối với điều phối viên, do tính chất công việc liên khoa nên được gán đồng thời cả ba compartment.

Mã	Viết tắt	Mô tả
10	TH	Khoa tiêu hóa
20	TK	Khoa thần kinh
30	TM	Khoa tim mạch

Group - Khu vực địa lý

Hệ thống bệnh viện hoạt động tại ba cơ sở: Hồ Chí Minh (HCM), Hải Phòng (HP) và Hà Nội (HN). Thông báo có thể được gửi đến một hoặc nhiều khu vực, và người dùng chỉ nhận được thông báo thuộc khu vực mình đang công tác.

Mã	Viết tắt	Mô tả
10	HCM	Hồ Chí Minh
20	HP	Hải Phòng
30	HN	Hà Nội

2.3.3. Giải pháp thực hiện

a. Tạo Policy và bảng THONGBAO

Nhóm tạo bảng THONGBAO gồm các trường MATB (mã thông báo), NOIDUNG, NGAYGIO và DIADIEM để lưu trữ nội dung thông báo nội bộ của bệnh viện. Tiếp theo, nhóm sử dụng SA_SYSDBA.CREATE_POLICY để tạo policy OLS có tên THONGBAO_OLS với cột nhãn OLS_LABEL trên bảng này. Policy được cấu hình với các tùy chọn READ_CONTROL, WRITE_CONTROL và CHECK_CONTROL, đảm bảo kiểm soát cả thao tác đọc lẫn ghi dữ liệu theo nhãn bảo mật.

Sau khi tạo policy, nhóm định nghĩa các thành phần nhãn bằng gói SA_COMPONENTS: ba Level (NV, LDK, BGD), ba Compartment (TH, TK, TM) và ba Group (HCM, HP, HN). Các data label cụ thể cũng được tạo thông qua SA_LABEL_ADMIN.CREATE_LABEL để sẵn sàng gán cho dữ liệu thông báo. Cuối cùng, policy được áp dụng lên bảng THONGBAO bằng SA_POLICY_ADMIN.APPLY_TABLE_POLICY.

Ngoài ra, nhóm xây dựng cơ chế fallback cho trường hợp môi trường Oracle chưa kích hoạt OLS. Khi phát hiện OLS không khả dụng, hệ thống tự động thêm cột OLS_LABEL kiểu VARCHAR2 vào bảng và tạo các hàm CHAR_TO_LABEL, LABEL_TO_CHAR giả lập để đảm bảo ứng dụng vẫn hoạt động bình thường.

b. Gán nhãn cho dữ liệu thông báo

Nhóm tạo 7 thông báo mẫu (T1 – T7) với các nhãn bảo mật khác nhau nhằm kiểm chứng cơ chế phân quyền đọc. Mỗi thông báo được gán nhãn thông qua hàm CHAR_TO_LABEL khi INSERT dữ liệu:

Mã	Nội dung tóm tắt	Nhãn OLS	Ý nghĩa
T1	Thông báo gửi toàn bộ nhân viên	NV	Mọi nhân viên đều đọc được

T2	Thông báo dành cho Ban giám đốc	BGD	Chỉ Ban giám đốc đọc được
T3	Thông báo dành cho lãnh đạo khoa	LDK	Lãnh đạo khoa và BGD đọc được
T4	Thông báo cho lãnh đạo Khoa tiêu hóa	LDK:TH	LDK thuộc Khoa tiêu hóa
T5	Thông báo cho NV Khoa tiêu hóa tại HCM	NV:TH:HCM	NV Khoa TH ở HCM
T6	Thông báo cho NV Khoa tiêu hóa tại HN	NV:TH:HN	NV Khoa TH ở Hà Nội
T7	Thông báo cho LDK Khoa TH và TK tại HP	LDK:TH,TK:HP	LDK Khoa TH hoặc TK ở HP

c. Gán nhãn cho người dùng

Nhân người dùng được gán thông qua `SA_USER_ADMIN.SET_USER_LABELS` với các tham số xác định phạm vi đọc tối đa (`max_read_label`), phạm vi ghi tối đa (`max_write_label`), phạm vi ghi tối thiểu (`min_write_label`), nhãn mặc định (`def_label`) và nhãn dòng (`row_label`). Tài khoản ADMINHOS được gán quyền FULL để có thể thêm dữ liệu với mọi nhãn bảo mật.

Nhóm tạo 8 tài khoản demo (U1 - U8) đại diện cho các vị trí khác nhau trong bệnh viện để kiểm chứng:

User	Nhãn truy cập	Ý nghĩa
U1	BGD:TH,TK,TM:HCM,HP,HN	Giám đốc - đọc được tất cả
U2	LDK:TM:HCM	Lãnh đạo Khoa tim mạch tại HCM
U3	LDK:TK:HN	Lãnh đạo Khoa thần kinh tại HN
U4	NV:TK:HCM	Nhân viên Khoa thần kinh tại HCM
U5	NV:TM:HCM	Nhân viên Khoa tim mạch tại HCM
U6	LDK:TM:HCM	Lãnh đạo Khoa tim mạch tại HCM
U7	LDK:TH,TK,TM:HCM,HP,HN	Lãnh đạo toàn bộ khoa, toàn bộ cơ sở
U8	NV:TH:HN	Nhân viên Khoa tiêu hóa tại HN

Đối với các tài khoản nhân viên thực tế trong bảng NHANVIEN, nhóm xây dựng vòng lặp tự động ánh xạ chuyên khoa và cơ sở làm việc sang mã compartment và group tương ứng (ví dụ “Khoa tiêu hóa” → TH, “Hồ Chí Minh” → HCM), sau đó gán nhãn và cấp quyền SELECT trên view VW_THONGBAO_APP. Riêng điều phối viên được gán đồng thời cả ba compartment (TH, TK, TM) do tính chất công việc liên khoa.

d. Tạo View hỗ trợ giao diện

Nhóm tạo view VW_THONGBAO_APP trên bảng THONGBAO để cung cấp dữ liệu cho ứng dụng. View này sử dụng hàm LABEL_TO_CHAR(OLS_LABEL) để chuyển giá trị nhãn số thành chuỗi ký tự dễ đọc (ví dụ NV:TH:HCM). Khi người dùng truy vấn view, OLS tự động lọc và chỉ trả về những thông báo có nhãn phù hợp với nhãn của người dùng đang đăng nhập, mà không cần bất kỳ xử lý lọc nào ở tầng ứng dụng.

2.3.4. Kết quả đạt được

Sau khi triển khai, hệ thống OLS hoạt động đúng theo thiết kế. Mỗi người dùng khi đăng nhập và truy vấn view VW_THONGBAO_APP chỉ nhìn thấy các thông báo phù hợp với nhãn bảo mật của mình. Ví dụ, U1 (Ban giám đốc) đọc được toàn bộ 7 thông báo, trong khi U8 (Nhân viên Khoa tiêu hóa tại Hà Nội) chỉ đọc được T1 (thông báo toàn bộ nhân viên) và T6 (thông báo dành cho NV Khoa tiêu hóa tại Hà Nội). Cơ chế lọc diễn ra hoàn toàn tự động ở tầng cơ sở dữ liệu, ứng dụng không cần viết thêm logic phân quyền.

Việc áp dụng OLS cho bài toán phát tán thông báo đã đảm bảo tính bảo mật theo chiều sâu: thông báo nhạy cảm chỉ đến đúng đối tượng được phép, đồng thời giữ cho tầng ứng dụng đơn giản vì toàn bộ logic kiểm soát truy cập được xử lý bởi Oracle Database.

2.4. Yêu cầu 3 - Kiểm toán (Audit)

2.4.1. Cơ sở lý thuyết

Audit là cơ chế ghi lại các hoạt động xảy ra bên trong cơ sở dữ liệu nhằm mục đích giám sát, phát hiện hành vi bất thường và hỗ trợ điều tra khi có sự cố. Trong môi trường y tế, dữ liệu bệnh nhân, hồ sơ bệnh án và đơn thuốc là những thông tin cực kỳ nhạy cảm. Việc theo dõi ai đã xem, ai đã thay đổi gì và khi nào là yêu cầu bắt buộc để đảm bảo tính toàn vẹn và trách nhiệm giải trình trong hệ thống.

2.4.2. Standard Audit

a. Lý thuyết

Standard Audit là cơ chế được kích hoạt bằng câu lệnh AUDIT trực tiếp trong Oracle. Cách hoạt động: DBA chỉ định đối tượng cần theo dõi (bảng, view, stored procedure, function)

và loại hành vi cần ghi nhận (SELECT, UPDATE, INSERT, DELETE, EXECUTE). Oracle sẽ tự động tạo một bản ghi trong bảng hệ thống SYS.AUD\$ mỗi khi câu lệnh đó được thực thi.

Dữ liệu nhật ký từ Standard Audit có thể được truy xuất qua view DBA_AUDIT_TRAIL, trong đó lưu các thông tin như tên user thực thi, tên đối tượng bị thao tác, thao tác là gì, thời gian xảy ra, mã lỗi trả về (nếu có), và terminal mà user đang kết nối từ.

Hai tùy chọn quan trọng khi thiết lập Standard Audit:

- **BY ACCESS:** Ghi một bản ghi cho mỗi lần câu lệnh được thực thi. Đây là lựa chọn chi tiết và phù hợp khi cần biết chính xác có bao nhiêu lần thao tác xảy ra.
- **WHENEVER SUCCESSFUL** và **WHENEVER NOT SUCCESSFUL:** Cho phép lọc theo kết quả thực thi. **WHENEVER SUCCESSFUL** chỉ ghi những lần thành công; **WHENEVER NOT SUCCESSFUL** chỉ ghi những lần thất bại, ví dụ như user không có quyền nhưng vẫn cố truy cập.

b. Giải pháp thực hiện

Trước khi thiết lập các tình huống mới, nhóm chạy lệnh NOAUDIT để xóa cấu hình cũ (nếu có) và dùng DBMS_AUDIT_MGMT.CLEAN_AUDIT_TRAIL để làm sạch dữ liệu nhật ký từ lần chạy trước. Điều này đảm bảo kết quả kiểm chứng không bị trộn lẫn với dữ liệu cũ.

Nhóm đề xuất 5 tình huống kiểm toán phản ánh đúng các nghiệp vụ thực tế của hệ thống bệnh viện:

- Ngưỡng cảnh 1 - Theo dõi bác sĩ xem và cập nhật hồ sơ bệnh án thành công

```
AUDIT SELECT, UPDATE ON ADMINHOS.HSBA  
BY ACCESS  
WHENEVER SUCCESSFUL;
```

Bảng HSBA là trung tâm của toàn bộ hệ thống, lưu thông tin chẩn đoán, điều trị và kết luận của từng bệnh nhân. Việc biết ai đã đọc hoặc sửa hồ sơ bệnh án là yêu cầu tối thiểu trong mọi hệ thống y tế. Tình huống này ghi nhận mọi thao tác SELECT và UPDATE thành công, bao gồm cả bác sĩ hợp pháp lẫn các tài khoản khác đang có quyền.

- Ngưỡng cảnh 2 - Phát hiện cố ý truy cập trái phép vào hồ sơ bệnh án

```
AUDIT SELECT, UPDATE ON ADMINHOS.HSBA  
BY ACCESS  
WHENEVER NOT SUCCESSFUL;
```

Đây là tình huống giám sát bảo mật: nếu một user không có quyền (ví dụ kỹ thuật viên hoặc bệnh nhân) cố gắng SELECT hay UPDATE thẳng vào bảng HSBA, Oracle sẽ từ chối và

Standard Audit sẽ ghi lại sự kiện đó với RETURNCODE khác 0. Từ log này, DBA có thể phát hiện các tài khoản đang thử thăm dò dữ liệu ngoài phạm vi.

- **Ngữ cảnh 3 - Theo dõi nhân viên tự cập nhật thông tin cá nhân**

```
AUDIT SELECT, UPDATE ON ADMINHOS.VW_NHANVIEN_SELF  
BY ACCESS  
WHENEVER SUCCESSFUL;
```

Nhân viên (bác sĩ, kỹ thuật viên, điều phối viên) chỉ được phép xem và sửa thông tin của chính mình thông qua view VW_NHANVIEN_SELF. View này đã được thiết kế để lọc chỉ trả về dữ liệu của SESSION_USER. Tình huống này giám sát các thao tác trên giao diện quản lý hồ sơ cá nhân của nhân viên để biết tần suất sử dụng và phát hiện bất thường.

- **Ngữ cảnh 4 - Theo dõi bệnh nhân tự xem và cập nhật thông tin cá nhân**

```
AUDIT SELECT, UPDATE ON ADMINHOS.VW_BENHNHAN_SELF  
BY ACCESS  
WHENEVER SUCCESSFUL;
```

Tương tự ngữ cảnh 3, nhưng áp dụng cho bệnh nhân. View VW_BENHNHAN_SELF giúp bệnh nhân chỉ thấy dữ liệu của mình. Việc audit trên view này giúp ghi nhận mỗi lần bệnh nhân đăng nhập xem hoặc cập nhật thông tin liên lạc và tiền sử bệnh.

- **Ngữ cảnh 5 - Theo dõi việc gọi stored procedure và function nghiệp vụ**

```
AUDIT EXECUTE ON ADMINHOS.SP_CAPNHAT_KETLUAN_HSBA  
BY ACCESS WHENEVER SUCCESSFUL;  
AUDIT EXECUTE ON ADMINHOS.FN_DEM_DONTHUOC  
BY ACCESS WHENEVER SUCCESSFUL;  
AUDIT EXECUTE ON ADMINHOS.SP_CAPNHAT_KETLUAN_HSBA  
BY ACCESS WHENEVER NOT SUCCESSFUL;  
AUDIT EXECUTE ON ADMINHOS.FN_DEM_DONTHUOC  
BY ACCESS WHENEVER NOT SUCCESSFUL;
```

Nhóm tạo hai đối tượng phục vụ demo kiểm toán:

- FN_DEM_DONTHUOC(p_mahsba) : Hàm đếm số đơn thuốc của một hồ sơ bệnh án. Đây là nghiệp vụ đọc dữ liệu thường xuyên được bác sĩ dùng.
- SP_CAPNHAT_KETLUAN_HSBA(p_mahsba, p_ketluan) : Thủ tục cập nhật kết luận hồ sơ. Đây là thao tác ghi quan trọng, chỉ nên được thực hiện bởi bác sĩ phụ trách.

Tình huống này audit cả hai chiều: thành công (ghi nhận hoạt động bình thường) và thất bại (phát hiện user không có quyền EXECUTE nhưng cố gọi chương trình). Quyền EXECUTE chỉ được cấp cho BS0001 và BS0002 trong script, nên nếu một tài khoản khác cố gọi sẽ bị ghi nhận ngay.

c. Kết quả đạt được

Sau khi thiết lập, toàn bộ dữ liệu kiểm toán Standard Audit được tập trung tại view DBA_AUDIT_TRAIL.

Nhóm xây dựng stored procedure ADMINHOS.SP_XEM_AUDIT_LOG để đọc dữ liệu từ view này và trả về qua SYS_REFCURSOR. Ứng dụng WinForms gọi procedure này qua AuditLogDAO.GetAuditLogs() và hiển thị lên bảng dgvLogs ở màn hình Audit Log của Quản trị viên.

Trong màn hình, mỗi bản ghi Standard Audit được phân biệt rõ bằng nhãn STANDARD, mã tự tăng dạng STD-XXXXX. Kết quả được tô màu theo trạng thái: trắng cho thành công, đỏ nhạt cho thất bại, giúp DBA nhận biết ngay các sự kiện bất thường mà không cần đọc từng dòng chi tiết.

2.4.3. Fine-Grained Audit / Unified Audit

a. Lý thuyết

- **Unified Audit:** là cơ chế kiểm toán thế hệ mới được Oracle giới thiệu từ phiên bản 12c. Khác với Standard Audit và FGA - hai cơ chế lưu nhật ký riêng rẽ vào SYS.AUD\$ và SYS.FGA_LOG\$, Unified Audit gom toàn bộ dữ liệu kiểm toán về một điểm duy nhất là view UNIFIED_AUDIT_TRAIL, được lưu trong schema bảo mật AUDSYS.
 - + Unified Audit sử dụng mô hình policy rõ ràng hơn: DBA tạo policy bằng lệnh CREATE AUDIT POLICY và kích hoạt bằng AUDIT POLICY, thay vì dùng câu lệnh AUDIT trực tiếp như Standard Audit. Cách tổ chức này giúp quản lý các quy tắc kiểm toán theo từng nhóm có tên, dễ bật/tắt mà không cần viết lại từ đầu.
 - + Về bảo mật, schema AUDSYS được Oracle quản lý nội bộ và không thể bị xóa hay sửa bởi bất kỳ user nào kể cả SYS, đảm bảo tính toàn vẹn tuyệt đối của nhật ký. Đây là điểm khắc phục hạn chế của Standard Audit truyền thống, nơi bảng SYS.AUD\$ có thể bị xóa nếu DBA có quyền đủ cao.
- **Fine-Grained Audit:** là cơ chế kiểm toán nâng cao của Oracle cho phép theo dõi dữ liệu ở mức độ chi tiết hơn Standard Audit rất nhiều. Nếu Standard Audit chỉ ghi lại ai

đã làm gì với đối tượng nào, thì FGA ghi lại thêm dòng dữ liệu nào thỏa điều kiện và câu SQL đầy đủ tại thời điểm thao tác xảy ra.

- + FGA được quản lý thông qua gói DBMS_FGA. Nhật ký FGA được lưu trong bảng hệ thống SYS.FGA_LOG\$ và có thể truy vấn qua view DBA_FGA_AUDIT_TRAIL. Khác với Standard Audit ghi vào SYS.AUD\$, hai bảng log này hoàn toàn độc lập nên có thể dùng song song mà không xung đột.
- + FGA hoạt động dựa trên khái niệm *policy*. Mỗi policy được gắn vào một bảng hoặc view cụ thể, khai báo loại câu lệnh cần theo dõi (SELECT, INSERT, UPDATE, DELETE) và điều kiện để quyết định có ghi nhật ký hay không.

- Theo dõi ở mức chi tiết:

- + Standard Audit có hạn chế là chỉ biết thao tác nào đã xảy ra trên đối tượng nào, nhưng không biết cụ thể dữ liệu nào bị tác động. Fine-Grained Audit (FGA) giải quyết vấn đề này bằng cách cho phép đặt điều kiện kiểm toán xuống tới mức từng dòng và từng cột dữ liệu.
- + FGA hoạt động thông qua gói DBMS_FGA. Khi một câu lệnh SQL tác động vào bảng được bảo vệ, Oracle sẽ đánh giá điều kiện audit_condition trên từng dòng dữ liệu liên quan. Nếu điều kiện trả về TRUE cho ít nhất một dòng, Oracle tạo một bản ghi trong bảng hệ thống SYS.FGA_LOG\$, có thể truy vấn qua view DBA_FGA_AUDIT_TRAIL.
- + So với Standard Audit, FGA có những đặc điểm nổi bật:
 - Có thể đặt điều kiện nghiệp vụ phức tạp, kể cả gọi function bên trong audit_condition.
 - Có thể chỉ định chính xác cột nào cần theo dõi qua tham số audit_column, thay vì theo dõi toàn bộ bảng.
 - Khi dùng DBMS_FGA.DB + DBMS_FGA.EXTENDED, audit trail sẽ lưu luôn câu SQL đầy đủ (SQL_TEXT) và các giá trị tham số đầu vào, rất cần thiết khi cần truy vết chính xác nội dung thay đổi.

- Theo dõi theo điều kiện:

- + Tham số audit_condition là điểm mạnh nhất của FGA. Đây là một chuỗi biểu thức SQL điều kiện (predicate) được Oracle đánh giá tại thời điểm câu lệnh thực thi. Ví dụ:
 - 'SYS_CONTEXT(''USERENV'', ''SESSION_USER'') = MABS': điều kiện đúng khi user đang đăng nhập chính là bác sĩ phụ trách hồ sơ.

- 'ADMINHOS.FN_FGA_IS_DPV() = 0': gọi function helper để kiểm tra user hiện tại có phải điều phối viên không.
- + Khi audit_condition = NULL, Oracle hiểu là luôn audit (tương đương '1 = 1'). Tham số audit_column_opts kiểm soát cách Oracle so khớp với danh sách cột:
 - DBMS_FGA.ANY_COLUMNS (mặc định): audit nếu câu lệnh đụng vào bất kỳ cột nào trong danh sách.
 - DBMS_FGA.ALL_COLUMNS: chỉ audit khi câu lệnh đụng đến tất cả cột trong danh sách cùng lúc.

b. Giải pháp thực hiện

Nhóm tạo hai hàm helper trong schema ADMINHOS để tái sử dụng logic kiểm tra vai trò trong nhiều policy:

- FN_FGA_IS_BS_PHUTRACH_HSBA(p_mahsba): Kiểm tra xem SESSION_USER có phải là bác sĩ/y sĩ đang phụ trách hồ sơ p_mahsba không. Hàm trả về 1 nếu đúng, 0 nếu sai. Bên trong, hàm JOIN bảng HSBA với NHANVIEN và lọc theo MABS = SESSION_USER và VAITRO = 'Bác sĩ/Y sĩ'.
- FN_FGA_IS_DPV(): Kiểm tra xem SESSION_USER có phải điều phối viên không, bằng cách tra cứu trong bảng NHANVIEN với điều kiện VAITRO = 'Điều phối viên'.
- **Tình huống 1 - Audit cập nhật đơn thuốc bởi bác sĩ/y sĩ**

Policy: FGA_DONTHUOC_UPDATE trên bảng DONTHUOC.

```
DBMS_FGA.ADD_POLICY(
  object_schema => 'ADMINHOS',
  object_name   => 'DONTHUOC',
  policy_name   => 'FGA_DONTHUOC_UPDATE',
  audit_condition => 'SYS_CONTEXT(''USERENV'', ''SESSION_USER'')
LIKE ''BS%''',
  audit_column  => 'MAHSBA,NGAYDT,TENTHUOC,LIEUDUNG',
  statement_types => 'UPDATE',
  audit_trail    => DBMS_FGA.DB + DBMS_FGA.EXTENDED,
  audit_column_opts => DBMS_FGA.ANY_COLUMNS
);
```

- + Tình huống này giám sát việc bác sĩ/y sĩ chỉnh sửa đơn thuốc sau khi đã được tạo. Đơn thuốc là tài liệu y tế quan trọng; nếu bị thay đổi (đặc biệt là tên thuốc hay liều dùng) sau khi đã cấp cho bệnh nhân, đây là hành vi cần được ghi lại đầy đủ để truy vết sau này.
- + Điều kiện `SESSION_USER LIKE 'BS%'` xác định đây là thao tác từ tài khoản bác sĩ/y sĩ (đặt tên theo quy ước BS0001, BS0002,...).
- + Các cột `MAHSBA`, `NGAYDT`, `TENTHUOC`, `LIEUDUNG` là bốn trường cốt lõi của đơn thuốc; nếu bất kỳ cột nào trong số này bị cập nhật thì audit ghi nhận.
- + Tùy chọn `EXTENDED` đảm bảo câu SQL đầy đủ được lưu vào `DBA_FGA_AUDIT_TRAIL.SQL_TEXT`, giúp DBA biết chính xác giá trị nào đã được thay đổi.

- Tình huống 2 - Audit bác sĩ cập nhật hợp pháp hồ sơ bệnh án

Policy: `FGA_HSBA_UPDATE_HOPPHAP` trên bảng `HSBA`.

```
DBMS_FGA.ADD_POLICY(
  object_schema => 'ADMINHOS',
  object_name   => 'HSBA',
  policy_name   => 'FGA_HSBA_UPDATE_HOPPHAP',
  audit_condition => 'SYS_CONTEXT(''USERENV'', ''SESSION_USER'') = MABS',
  audit_column  => 'CHANDOAN,DIEUTRI,KETLUAN',
  statement_types => 'UPDATE',
  audit_trail    => DBMS_FGA.DB + DBMS_FGA.EXTENDED,
  audit_column_opts => DBMS_FGA.ANY_COLUMNS
);
```

Điều kiện `SESSION_USER = MABS` nghĩa là chỉ audit khi người đang đăng nhập chính là bác sĩ phụ trách hồ sơ đó (cột `MABS` trong `HSBA`). Đây là hành vi hợp pháp: bác sĩ BS0001 đang cập nhật chẩn đoán cho bệnh nhân mà mình phụ trách.

- Tình huống 3 - Audit cập nhật bất hợp pháp hồ sơ bệnh án

Policy: `FGA_HSBA_UPDATE_BATHOPPHAP` trên bảng `HSBA`.

```
DBMS_FGA.ADD_POLICY(
  object_schema => 'ADMINHOS',
  object_name   => 'HSBA',
```

```

policy_name      => 'FGA_HSBA_UPDATE_BATHOPPHAP',
audit_condition  => 'SYS_CONTEXT(''USERENV'', ''SESSION_USER'') <>
NVL(MABS, ''#'')',
audit_column     => 'CHANDOAN,DIEUTRI,KETLUAN',
statement_types => 'UPDATE',
audit_trail      => DBMS_FGA.DB + DBMS_FGA.EXTENDED,
audit_column_opts => DBMS_FGA.ANY_COLUMNS
);

```

- + Đây là tình huống đối nghịch với TH2. Điều kiện `SESSION_USER <> NVL(MABS, '#')` bắt đúng trường hợp bác sĩ đang cố sửa hồ sơ của bệnh nhân không thuộc phạm vi phụ trách của mình. Hàm `NVL(MABS, '#')` xử lý trường hợp MABS là NULL (hồ sơ chưa được phân công bác sĩ): khi đó NVL trả về '#', đảm bảo điều kiện luôn đúng và hành vi vẫn bị audit.
- + Trong thực tế, VPD đã ngăn bác sĩ nhìn thấy dữ liệu ngoài phạm vi. Nhưng nếu bác sĩ cố UPDATE thẳng vào bảng (bỏ qua giao diện), FGA sẽ ghi lại ngay lập tức vì câu lệnh đã “chạy tới bảng” và điều kiện audit được đánh giá.

- **Tình huống 4 - Audit thao tác bất hợp pháp trên dịch vụ hỗ trợ chẩn đoán**

Đây là tình huống phức tạp, được tách thành 5 policy riêng biệt để xử lý từng loại hành vi trên bảng HSBA_DV:

+ **TH4a. INSERT bất hợp pháp**

Policy FGA_HSBA_DV_INSERT_BATHOPPHAP:

```

...
audit_condition => 'ADMINHOS.FN_FGA_IS_BS_PHUTRACH_HSBA(MAHSBA) = 0',
statement_types => 'INSERT'

```

Audit khi user thêm dịch vụ vào hồ sơ mà mình không phụ trách. Hàm `FN_FGA_IS_BS_PHUTRACH_HSBA(MAHSBA)` kiểm tra dựa trên mã hồ sơ (MAHSBA) của dòng đang được INSERT.

+ **TH4b. UPDATE cột MAKTV bất hợp pháp**

Policy FGA_HSBA_DV_UPDATE_MAKTV_BATHOPPHAP:

```

...
audit_condition  => 'ADMINHOS.FN_FGA_IS_DPV() = 0',

```

```
audit_column      => 'MAKTV',  
statement_types   => 'UPDATE'
```

Chỉ điều phối viên mới được phép gán kỹ thuật viên (MAKTV). Policy audit: nếu user không phải DPV (hàm trả về 0) thì ghi log cảnh báo.

+ **TH4c. UPDATE cột KETQUA bất hợp pháp**

Policy FGA_HSBA_DV_UPDATE_KETQUA_BATHOPPHAP:

```
...  
audit_condition    => 'SYS_CONTEXT(''USERENV'', ''SESSION_USER'') <>  
NVL(MAKTV, ''#'')',  
audit_column       => 'KETQUA',  
statement_types    => 'UPDATE'
```

Chỉ kỹ thuật viên được phân công cho dòng đó (tức SESSION_USER = MAKTV) mới được cập nhật kết quả. Nếu ai khác, kể cả một KTV không được phân công, cố sửa KETQUA thì sẽ bị ghi audit lại.

+ **TH4d. UPDATE các cột cốt lõi luôn bất hợp pháp**

Policy FGA_HSBA_DV_UPDATE_OTHER_BATHOPPHAP:

```
...  
audit_condition     => '1 = 1',  
audit_column        => 'MAHSBA, LOAIDV, NGAYDV',  
statement_types     => 'UPDATE'
```

Các cột MAHSBA, LOAIDV, NGAYDV là thông tin định danh của dịch vụ; không ai được phép thay đổi sau khi đã tạo. Điều kiện 1 = 1 nghĩa là audit mọi trường hợp, bất kể user là ai.

+ **TH4e. DELETE bất hợp pháp**

Policy FGA_HSBA_DV_DELETE_BATHOPPHAP:

```
...  
audit_condition => 'ADMINHOS.FN_FGA_IS_BS_PHUTRACH_HSBA(MAHSBA) = 0',  
statement_types => 'DELETE'
```

Chỉ bác sĩ/y sĩ phụ trách mới được xóa dòng dịch vụ khỏi hồ sơ bệnh án do mình đảm nhận.

c. Kết quả đạt được

Toàn bộ 8 policy FGA (1 cho DONTUOC, 2 cho HSBA, 5 cho HSBA_DV) đều được tạo thành công, xác nhận qua DBMS_OUTPUT.

Về nhật ký kiểm toán, dữ liệu FGA được tập trung đọc qua view DBA_FGA_AUDIT_TRAIL trong stored procedure SP_XEM_AUDIT_LOG. Procedure này sử dụng UNION ALL để ghép dữ liệu Standard Audit và FGA thành một kết quả thống nhất, trong đó:

- Cột LOAI_AUDIT phân biệt nguồn với giá trị 'STANDARD' hoặc 'FGA'.
- Cột KET_QUA gán giá trị 'Canh bao' cho các policy có tên chứa BATHOPPHAP (cảnh báo truy cập bất hợp pháp), các policy còn lại được gán 'Ghi nhan FGA'.
- Cột SQL_TEXT lưu câu SQL đầy đủ mà người dùng đã thực thi, chỉ có ở FGA khi kích hoạt DBMS_FGA.EXTENDED.

Ứng dụng WinForms đọc dữ liệu thông qua AuditLogDAO.GetAuditLogs() và hiển thị trên bảng dgvLogs. Giao diện cung cấp các bộ lọc theo loại audit (STANDARD/FGA), đối tượng, hành vi, kết quả và khoảng thời gian, giúp DBA thu hẹp phạm vi tìm kiếm nhanh chóng. Ngoài ra, dashboard của Quản trị viên hiển thị tổng số bản ghi audit và tổng số cảnh báo bất thường thông qua SP_GET_DASHBOARD_QTV, giúp DBA nắm bắt tình hình bảo mật ngay khi mở ứng dụng mà không cần vào màn hình nhật ký.

2.5. Yêu cầu 4 - Sao lưu và phục hồi

2.5.1. Cơ sở lý thuyết

Trong quản trị cơ sở dữ liệu hiện đại, sao lưu và phục hồi là hai tiến trình cốt lõi nhằm bảo toàn tài nguyên thông tin, ngăn ngừa thảm họa mất mát dữ liệu và bảo đảm tính liên tục của hệ thống nghiệp vụ.

A. Khái niệm Sao lưu Dữ liệu (Data Backup)

Sao lưu dữ liệu được định nghĩa là quy trình nhân bản và lưu trữ có hệ thống các cấu phần cấu thành cơ sở dữ liệu bao gồm các tập tin dữ liệu vật lý, tập tin kiểm soát cấu trúc và nhật ký lưu vết giao dịch đến một phân vùng lưu trữ độc lập. Tiến trình này nhằm mục đích thiết lập các rào cản dự phòng để bảo toàn tính toàn vẹn của thông tin trước các sự cố phần cứng, lỗi phần mềm hoặc các tác động chủ quan từ con người.

- **Phân loại theo trạng thái hoạt động của Hệ quản trị Cơ sở dữ liệu (Database State):**
 - + Sao lưu nóng (Online/Hot Backup): Tiến trình sao lưu được thực thi song song khi cơ sở dữ liệu đang trong trạng thái vận hành và phục vụ các giao dịch thời gian thực của người dùng. Điều kiện tiên quyết để áp dụng phương pháp này là cơ sở dữ liệu

phải cấu hình hoạt động trong chế độ ghi nhật ký lưu trữ, đảm bảo ghi nhận liên tục mọi biến động dữ liệu xảy ra trong quá trình sao lưu.

- + Sao lưu nguội (Offline/Cold Backup): Tiến trình được thực hiện sau khi cơ sở dữ liệu đã được dừng hoạt động một cách nhất quán. Tại trạng thái này, các tệp tin dữ liệu tĩnh tuyệt đối, bảo đảm tính đồng bộ toàn vẹn mà không cần phụ thuộc vào cơ chế đồng bộ giao dịch.
- **Phân loại theo bản chất của dữ liệu lưu trữ (Data Nature):**
 - + Sao lưu vật lý (Physical Backup): Tập trung vào việc sao chép nguyên trạng các tệp tin vật lý cấu thành nên cơ sở dữ liệu trên hệ điều hành. Đây là nền tảng cho việc khôi phục thảm họa phần cứng.
 - + Sao lưu logic (Logical Backup): Thực hiện trích xuất dữ liệu cấu trúc (Metadata) và các bản ghi lưu trữ bên trong các đối tượng (như bảng dữ liệu, lược đồ - Schema) thành các tệp tin kết xuất trung gian (Dump Files) để phục vụ việc chuyển đổi hoặc tái cấu trúc dữ liệu.

B. Khái niệm Phục hồi Dữ liệu (Data Recovery)

Phục hồi dữ liệu là tiến trình đưa hệ thống cơ sở dữ liệu trở lại trạng thái hoạt động bình thường, nhất quán và đáng tin cậy sau khi xảy ra các sự cố phá hoại vật lý hoặc logic. Tiến trình khôi phục toàn diện trải qua hai giai đoạn kế tiếp nhau:

- Giai đoạn Tái cấu trúc (Restore): Thực hiện việc định vị và sao chép các tệp tin vật lý từ các bản ghi lưu trữ dự phòng về lại các phân vùng đĩa cứng trên máy chủ.
- Giai đoạn Đồng bộ hóa (Recover): Sử dụng dữ liệu từ nhật ký giao dịch tuần tự (Redo Logs và Archive Logs) để thực hiện cơ chế chuyển tiếp giao dịch, nhằm áp dụng lại toàn bộ các thay đổi hợp lệ đã diễn ra từ thời điểm sao lưu gần nhất đến trước thời điểm xảy ra sự cố, ngăn ngừa tình trạng thất thoát thông tin.
 - + Khôi phục hoàn toàn (Complete Recovery): Khôi phục toàn bộ trạng thái dữ liệu đến thời điểm cuối cùng trước khi hệ thống ngưng hoạt động.
 - + Khôi phục không hoàn toàn / Phục hồi điểm thời gian (Incomplete Recovery / Point-in-Time Recovery - PITR): Phục hồi trạng thái dữ liệu về một mốc thời điểm hoặc mã chỉ số giao dịch (SCN - System Change Number) cụ thể được xác định trước trong quá khứ.

C. Các phương pháp sao lưu và phục hồi & phân tích ưu, nhược điểm

Hệ quản trị Oracle cung cấp ba nhóm cơ chế sao lưu/phục hồi chính. Bảng dưới đây so sánh bản chất, phạm vi và ưu - nhược điểm của từng phương pháp.

- ***C1 - RMAN (Recovery Manager) - Sao lưu/phục hồi vật lý***

Tiêu chí	Nội dung
Bản chất	Sao lưu vật lý: datafile, control file, archived redo log
Phạm vi	Toàn bộ cơ sở dữ liệu; hỗ trợ sao lưu toàn phần (Full) và gia tăng (Incremental)
Phục hồi	Khôi phục thảm họa phần cứng; hỗ trợ PITR chính xác đến mốc thời gian/SCN
Ưu điểm	Mạnh và toàn diện nhất; có PITR; sao lưu gia tăng tiết kiệm dung lượng; nén và kiểm tra tính hợp lệ của bản sao lưu; tự quản lý qua catalog/control file
Nhược điểm	Nặng và phức tạp; lệnh chạy từ dòng lệnh hệ điều hành, không gọi được từ ứng dụng/PL/SQL; khi phục hồi vật lý phải đưa CSDL về trạng thái giới hạn (MOUNT) — gián đoạn dịch vụ; bắt buộc bật ARCHIVELOG

- ***C2 - Oracle Data Pump (expdp / impdp, gói DBMS_DATAPUMP) - Sao lưu/phục hồi logic***

Tiêu chí	Nội dung
Bản chất	Sao lưu logic: kết xuất metadata + dữ liệu hàng của lược đồ ra tệp .dmp
Phạm vi	Linh hoạt ở mức toàn bộ / lược đồ (schema) / từng bảng
Phục hồi	Nạp lại dữ liệu vào CSDL (impdp); di chuyển dữ liệu giữa các hệ thống/phiên bản
Ưu điểm	Có API PL/SQL (DBMS_DATAPUMP) nên gọi trực tiếp được từ ứng dụng; lập lịch tự động được; chạy nền bất đồng bộ; cô lập đúng schema nghiệp vụ; tương thích đa phiên bản
Nhược điểm	Là ảnh chụp tại một thời điểm nên không hỗ trợ PITR; tiến trình impdp có thể chậm với dữ liệu lớn; không cứu được hỏng hóc vật lý cấp datafile

- ***C3 - Oracle Flashback (Query / Table / Drop / Database) - Phục hồi logic dựa trên UNDO***

Tiêu chí	Nội dung
Bản chất	Đọc dữ liệu quá khứ từ phân vùng UNDO và recycle bin
Phạm vi	Dòng / bảng / toàn CSDL
Phục hồi	Hoàn tác nhanh lỗi logic của người dùng như UPDATE/DELETE/DROP nhằm mà không cần khôi phục từ tệp sao lưu
Ưu điểm	Rất nhanh; nhắm đúng dòng/bảng cần khôi phục; không tạo tệp sao lưu; kết hợp được với nhật ký kiểm toán để khôi phục đúng thời điểm sự cố
Nhược điểm	Phụ thuộc cửa sổ lưu trữ UNDO (UNDO_RETENTION); quá hạn sẽ báo lỗi ORA-01555; không cứu được hỏng hóc vật lý

Nhận xét tổng quát: Không có phương pháp đơn lẻ nào là đủ. RMAN bảo vệ ở tầng vật lý cho thảm họa hạ tầng; Data Pump linh hoạt cho sao lưu/di chuyển logic ở mức lược đồ và có thể tự động hóa, gọi từ ứng dụng; Flashback xử lý nhanh các lỗi logic trong cửa sổ thời gian gần. Một chiến lược phòng vệ tốt cần phối hợp cả ba theo nguyên tắc phòng vệ nhiều lớp.

D. Giải pháp lựa chọn triển khai và lý do

Dựa trên đặc thù của hệ thống HospitalX, một ứng dụng client cần quản trị viên thao tác sao lưu/phục hồi trực tiếp trên giao diện, đồng thời cần bảo vệ trọng tâm là dữ liệu nghiệp vụ y tế thuộc lược đồ ADMINHOS. Nhóm lựa chọn mô hình tích hợp đa tầng như sau:

Hạng mục	Phương pháp lựa chọn	Lý do
Sao lưu chủ động & tự động	Oracle Data Pump (DBMS_DATAPUMP)	Là cơ chế duy nhất có API PL/SQL để đóng gói thành thủ tục lưu trữ và gọi xuống trực tiếp từ ứng dụng; cho phép lập lịch tự động và chạy nền; có lập đúng lược đồ nghiệp vụ.
Phục hồi từ bản sao lưu	Data Pump Import (impdp)	Vì bản sao lưu là tệp .dmp logic, công nghệ phục hồi tương ứng là impdp; nạp lại đúng các bảng dữ liệu y tế, giữ nguyên cấu trúc và toàn bộ lớp bảo mật như VPD/OLS/trigger/view.
Khôi phục sau sự cố bảo mật	Flashback theo nhật ký kiểm toán (kết hợp Yêu cầu 3)	Dùng mốc thời gian trong nhật ký kiểm toán để hoàn tác đúng các dòng bị tác động trái phép, không ảnh hưởng dữ liệu hợp lệ.
Lớp dự phòng thảm họa	RMAN (ở mức tài liệu hóa quy trình)	Khôi phục vật lý đòi hỏi quyền quản trị tối cao và phải đưa CSDL về trạng thái MOUNT nên không phù hợp gọi từ ứng dụng; được chuẩn hóa thành quy trình lệnh để vận hành thủ công khi xảy ra hỏng hóc vật lý.

Kết luận lựa chọn: Data Pump được chọn làm trục chính cho cả sao lưu và phục hồi trong ứng dụng vì khả năng lập trình hóa (gọi từ C# qua thủ tục lưu trữ), tự động hóa (lập lịch) và đúng phạm vi (lược đồ nghiệp vụ). Flashback và RMAN đóng vai trò các lớp hỗ trợ cho từng tình huống chuyên biệt.

2.5.2. Giải pháp thực hiện

Dựa trên cấu trúc hạ tầng cơ sở dữ liệu và yêu cầu nghiệp vụ đặc thù của hệ thống quản lý bệnh viện HospitalX, nhóm thực hiện đã thiết kế và triển khai mô hình sao lưu và phục hồi

tích hợp đa tầng, kết hợp đồng bộ giữa các tác vụ quản trị tại nhân hệ quản trị cơ sở dữ liệu và giao diện điều hành tương tác trực quan của quản trị viên trên ứng dụng client.

A. Giải pháp sao lưu logic tự động và chủ động bằng công nghệ Oracle Data Pump

Đây là giải pháp cốt lõi được ứng dụng vận hành và điều khiển trực tiếp nhằm bảo vệ toàn bộ dữ liệu nghiệp vụ y tế thuộc lược đồ của tài khoản quản trị nghiệp vụ.

Tiến trình sao lưu logic chủ động: Hệ thống xây dựng thủ tục lưu trữ chuyên dụng sử dụng gói thư viện hệ thống DBMS_DATAPUMP của Oracle để kết xuất toàn bộ dữ liệu cấu trúc và dữ liệu bảng nghiệp vụ. Tiến trình này được liên kết trực tiếp với nút chức năng trên giao diện điều hành của Quản trị viên, cho phép thực thi tác vụ sao lưu tức thời bất kỳ lúc nào dưới dạng tiến trình chạy ngầm bất đồng bộ.

Tiến trình sao lưu logic tự động định kỳ: Nhóm thực hiện thiết lập trình lập lịch tác vụ hệ thống (DBMS_SCHEDULER) để tự động hóa hoàn toàn quy trình này. Tác vụ được đặt lịch chạy tự động định kỳ vào lúc 02:00 AM hàng ngày để kết xuất dữ liệu và ghi nhận tệp tin nhật ký kết quả vào phân vùng lưu trữ bảo mật trên máy chủ, đảm bảo tính liên tục của dữ liệu mà không cần sự can thiệp thủ công từ con người.

B. Quy trình sao lưu vật lý và mô phỏng phục hồi bằng giải pháp Oracle RMAN

- ***B1 - Phục hồi logic thực tế bằng Data Pump Import***

Hệ thống triển khai phục hồi logic thực tế bằng Data Pump Import ngay trên giao diện ứng dụng. Khi quản trị viên chọn một bản sao lưu hợp lệ và yêu cầu phục hồi, hệ thống thực hiện:

1. Xác thực bản sao lưu: tra cứu thông tin tệp `.dmp` của bản sao lưu được chọn từ bảng nhật ký, chỉ chấp nhận bản ở trạng thái thành công.
2. Chuẩn bị toàn vẹn tham chiếu: tạm thời vô hiệu hóa các ràng buộc khóa ngoại để tránh xung đột thứ tự khi nạp đè dữ liệu.
3. Nạp lại dữ liệu (Import): dùng DBMS_DATAPUMP ở chế độ IMPORT, đọc tệp .dmp, xóa sạch dữ liệu cũ và nạp lại dữ liệu của các bảng nghiệp vụ (tùy chọn TABLE_EXISTS_ACTION = TRUNCATE). Cách làm này giữ nguyên cấu trúc bảng và toàn bộ lớp bảo mật (VPD, OLS, trigger, view) đã thiết lập, chỉ thay phần dữ liệu hàng.
4. Khôi phục ràng buộc: bật lại các khóa ngoại sau khi nạp xong (kể cả khi có lỗi).
5. Ghi nhận kết quả: cập nhật bảng nhật ký phục hồi và trả về trạng thái/thông điệp để hiển thị cho quản trị viên.

Tiến trình phục hồi chạy ở luồng nền, tiến độ được hiển thị trực quan theo từng phân đoạn công việc trên bảng điều khiển của giao diện.

- **B2 - Khôi phục dữ liệu dựa trên nhật ký kiểm toán (sự cố bảo mật)**

Ngoài việc phục hồi từ bản sao lưu, hệ thống còn triển khai một cơ chế khôi phục chuyên biệt cho sự cố bảo mật - nhằm đáp ứng trực tiếp yêu cầu “khôi phục dữ liệu dựa vào nhật ký kiểm toán ở Yêu cầu 3” (sau khi có sự cố)”. Cơ chế này kết hợp nhật ký kiểm toán mịn (FGA) của Yêu cầu 3 với kỹ thuật Flashback Query của Oracle.

Nguyên lý: Cơ chế giám sát FGA đã ghi nhận mọi hành vi can thiệp trái phép lên các bảng hồ sơ bệnh án, dịch vụ kỹ thuật và đơn thuốc, kèm theo dấu thời gian chính xác của từng sự cố. Hệ thống tận dụng chính dấu thời gian này làm "tọa độ thời gian" để đưa các dòng dữ liệu bị tác động quay về trạng thái ngay trước thời điểm xảy ra sự cố. Như vậy, nhật ký vốn chỉ phục vụ giám sát (Yêu cầu 3) được nâng cấp thành đầu vào cho công cụ phục hồi (Yêu cầu 4), gắn kết hai yêu cầu thành một quy trình "phát hiện → khôi phục" liền mạch.

Quy trình thực hiện:

1. Truy vết thời điểm sự cố: đọc nhật ký kiểm toán, lọc các bản ghi ứng với hành vi bất hợp pháp, xác định thời điểm xảy ra sự cố sớm nhất (bỏ qua các thao tác do chính tài khoản quản trị thực hiện).
2. Xác định mốc dữ liệu sạch: lùi về một giây trước thời điểm sự cố và quy đổi sang mã giao dịch SCN (System Change Number), định danh chính xác một phiên bản dữ liệu trong quá khứ.
3. Đọc dữ liệu nguyên bản: dùng Flashback Query (truy vấn dữ liệu “tại mốc SCN”) để lấy lại bản gốc của các dòng bị tác động từ phân vùng UNDO.
4. Hoàn tác sự cố: đối chiếu dữ liệu hiện tại với bản gốc và xử lý cả ba loại tác động trái phép - hoàn tác sửa đổi (cập nhật lại giá trị cũ), hoàn tác xóa (chèn lại dòng đã mất), và hoàn tác chèn (loại bỏ dòng được thêm trái phép). Việc xử lý được thực hiện theo thứ tự an toàn khóa ngoại giữa các bảng và chỉ tác động đến đúng những dòng bị thay đổi.
5. Ghi nhận kết quả: lưu nhật ký khôi phục và trả về số dòng đã xử lý cùng trạng thái để hiển thị cho quản trị viên.

Tiến trình được kích hoạt bán tự động: quản trị viên xác nhận và phát lệnh từ giao diện kiểm toán, toàn bộ các bước truy vết - tính toán - hoàn tác do hệ thống tự thực hiện.

2.5.3. Kết quả đạt được

Thông qua giao diện quản trị tương tác trực quan được xây dựng trên ứng dụng và các kịch bản thực nghiệm trực tiếp trên hệ cơ sở dữ liệu, giải pháp đã đạt được các kết quả thực tế như sau:

Kết quả sao lưu thành công: Hệ thống cho phép thực hiện việc kiểm thử giả định cấu hình trước khi chạy thực tế để đánh giá tính khả thi của tiến trình. Khi lệnh sao lưu chủ động được phát động, tiến trình sao lưu logic được khởi chạy thành công dưới nền hệ quản trị cơ sở dữ liệu, các tệp tin kết xuất dữ liệu và tệp tin ghi vết tiến trình được tạo ra đầy đủ tại phân vùng lưu trữ quy định. Đồng thời trên giao diện, lịch sử sao lưu được cập nhật tức thì với trạng thái vận hành hoàn hảo để quản trị viên theo dõi trực quan.

Kết quả phục hồi thành công: Tiến trình phục hồi bằng Data Pump Import được thực thi thật trên cơ sở dữ liệu: xác thực bản sao lưu nguồn, nạp lại dữ liệu các bảng nghiệp vụ về đúng trạng thái tại thời điểm sao lưu, đồng thời bảo toàn toàn vẹn cấu trúc và các chính sách bảo mật. Nhật ký bảng điều khiển ghi nhận tường minh tiến độ từng phân đoạn; dữ liệu sau khôi phục đạt trạng thái nhất quán và hệ thống tiếp tục vận hành bình thường.

2.6. Đánh giá phân hệ

Phân hệ 2 đã xây dựng ứng dụng quản lý dữ liệu y tế trên Oracle Database, đáp ứng các yêu cầu chính về cấp quyền truy cập, bảo vệ thông báo, kiểm toán và sao lưu – phục hồi dữ liệu. Hệ thống sử dụng RBAC cho các nhóm người dùng có tập quyền giống nhau như Bệnh nhân và Kỹ thuật viên; VPD để giới hạn dữ liệu theo từng người dùng đối với Điều phối viên và Bác sĩ/Y sĩ; OLS để kiểm soát thông báo theo nhãn bảo mật; Audit để ghi nhận và truy vết hành vi; đồng thời kết hợp Data Pump và RMAN cho bài toán sao lưu, phục hồi.

2.6.1. Ưu điểm

- **Bao phủ đầy đủ yêu cầu chức năng:** Phân hệ đã hiện thực các nhóm chức năng chính gồm cấp quyền truy cập, phát tán thông báo bằng OLS, kiểm toán hoạt động và sao lưu - phục hồi dữ liệu.
- **Áp dụng phù hợp nhiều cơ chế bảo mật của Oracle:** RBAC được dùng để quản lý quyền theo vai trò, VPD dùng để kiểm soát dữ liệu mức dòng, OLS dùng để kiểm soát thông báo theo nhãn, còn Audit dùng để giám sát và truy vết thao tác.
- **Đảm bảo nguyên tắc quyền tối thiểu:** Người dùng chỉ được truy cập và thao tác trên dữ liệu cần thiết cho vai trò của mình. Bệnh nhân chỉ thấy dữ liệu cá nhân, kỹ thuật viên chỉ xử lý dịch vụ được phân công, bác sĩ chỉ thao tác trên hồ sơ phụ trách.

- **Gắn bảo mật với nghiệp vụ y tế:** Các chính sách VPD, OLS và FGA được thiết kế theo các tình huống thực tế của bệnh viện như quản lý hồ sơ bệnh án, phân công kỹ thuật viên, cập nhật đơn thuốc và gửi thông báo theo cấp bậc, chuyên khoa, khu vực.
- **Có khả năng giám sát và phục hồi:** Nhật ký Standard Audit và FGA được hiển thị trên giao diện quản trị, giúp DBA theo dõi các hành vi thành công hoặc thất bại. Bên cạnh đó, cơ chế sao lưu bằng Data Pump và RMAN giúp tăng khả năng bảo vệ dữ liệu khi xảy ra sự cố.

2.6.2. Hạn chế

- Một số chính sách nhận diện vai trò người dùng dựa trên quy ước đặt tên tài khoản như BS, KTV, BN hoặc DP.
 - + Ảnh hưởng: Cách làm này phù hợp với dữ liệu demo nhưng có thể gây khó mở rộng trong hệ thống thực tế. Nếu tài khoản được đặt không đúng quy ước, policy có thể nhận diện sai vai trò người dùng.
 - + Hướng khắc phục: Xác định vai trò dựa trên dữ liệu trong bảng NHANVIEN, BENHNHAN hoặc role Oracle thay vì phụ thuộc vào tiền tố username.
- Việc kết hợp nhiều cơ chế bảo mật như RBAC, VPD, OLS và FGA làm hệ thống có độ phức tạp cao.
 - + Ảnh hưởng: Khi người dùng không xem được dữ liệu hoặc thao tác bị từ chối, DBA có thể mất nhiều thời gian để xác định nguyên nhân đến từ quyền đối tượng, VPD policy, OLS label hay audit policy.
 - + Hướng khắc phục: Bổ sung tài liệu mô tả chi tiết từng policy, xây dựng màn hình kiểm tra quyền hiệu lực và hiển thị thông báo lỗi rõ ràng hơn trên giao diện.
- Oracle Label Security yêu cầu môi trường Oracle được cấu hình đúng và tài khoản cài đặt có đủ quyền cần thiết.
 - + Ảnh hưởng: Nếu OLS chưa được bật hoặc user chưa được cấp quyền phù hợp, quá trình tạo policy, label và gán nhãn người dùng có thể không thực hiện được.
 - + Hướng khắc phục: Chuẩn bị script kiểm tra môi trường trước khi cài đặt, đồng thời bổ sung hướng dẫn cấp quyền và cấu hình OLS trong tài liệu triển khai.

TÀI LIỆU THAM KHẢO

- [1] Phạm, T. B. H., & Lương, V. M. (n.d.). *Lecture slides* [Course materials]. Information System Department, Faculty of Information Technology, University of Science, VNU-HCM.
- [2] Oracle. (2024). *Oracle Database Security Guide 21c*. Oracle Corporation. Retrieved June 17, 2026, from <https://docs.oracle.com/en/database/oracle/oracle-database/21/dbseg/>
- [3] Oracle. (2024). *Oracle Database SQL Language Reference 21c*. Oracle Corporation. Retrieved June 17, 2026, from <https://docs.oracle.com/en/database/oracle/oracle-database/21/sqlrf/>
- [4] Oracle. (2024). *Oracle Database PL/SQL Language Reference 21c*. Oracle Corporation. Retrieved June 17, 2026, from <https://docs.oracle.com/en/database/oracle/oracle-database/21/lpls/>
- [5] Oracle. (2024). *Oracle Data Provider for .NET Developer's Guide 21c*. Oracle Corporation. Retrieved June 17, 2026, from <https://docs.oracle.com/en/database/oracle/oracle-database/21/odpnt/>
- [6] Kim, D., & Solomon, M. G. (2021). *Fundamentals of information systems security* (4th ed.). Jones & Bartlett Learning.
- [7] Oracle. (2024). *Oracle Database PL/SQL Packages and Types Reference 21c (DBMS_FGA)*. Oracle Corporation. Retrieved June 23, 2026, from https://docs.oracle.com/en/database/oracle/oracle-database/21/arpls/DBMS_FGA.html
- [8] Oracle. (2024). *Introduction to Auditing — Oracle Database Security Guide 21c*. Oracle Corporation. Retrieved June 23, 2026, from <https://docs.oracle.com/en/database/oracle/oracle-database/21/dbseg/introduction-to-auditing.html>